education

Информационный поиск

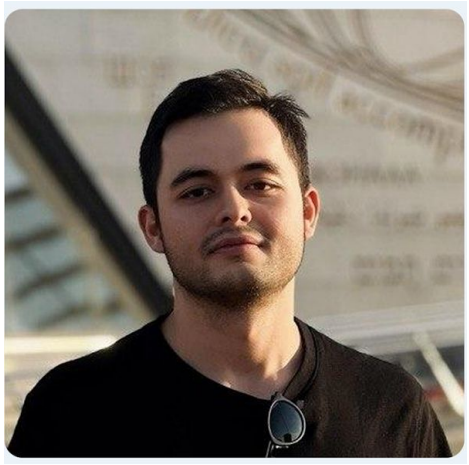
Преподаватель Руслан Аллаяров

Ведущий лекции



- Более 7 лет в ML, 6 лет из них в поиске
- Ex VK (teamlead поиска), Ex OKKO (teamlead рекома)
- Отвечал за блендинг поиска по видео, за базовое качество поиска по клипам и плейлистам
- Закончил мехмат МГУ и Техносферу (2021)

Команда курса: Преподаватели



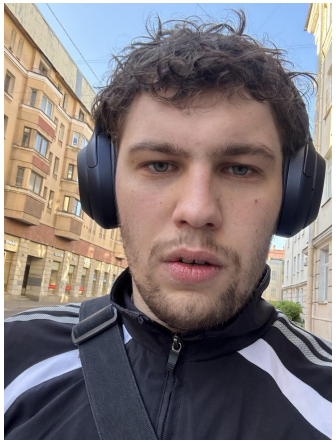
Руслан Аллаяров

Ex. руководитель группы специальных
сценариев поиска, VK AI
Эксперт-преподаватель VK



Николай Трофимов

Разработчик группы специальных сценариев поиска, VK AI



Анатолий Васильев

Разработчик группы базового качества поиска по видео, VK AI

Актуальность курса

Актуальность курса

- Сфера информационного поиска стремительно развивается благодаря глубокому обучению и большим языковым моделям.
- Растет спрос на специалистов, владеющих современными технологиями поиска и ранжирования.
- Курс сочетает классические подходы (TF-IDF, BM25) и передовые нейросетевые технологии (векторный поиск, BERT, DSSM).
- Эти технологии — стандарт индустрии и ключ к построению эффективных поисковых систем

Программа и логика курса

Курс состоит из:



13 онлайн-занятий



4 домашних задания



финальный проект

→ Курс проходит с 9 сентября по 9 декабря 2026 г. Занятия по средам, 18:10–21:00.

Программа курса:

Часть I: инженерная

- Введение
- Индексация и булев поиск
- Предобработка запросов. Исправление опечаток

Часть II: ранжирование

- Текстовое ранжирование I: модель векторного пространства
- Текстовое ранжирование II: метрики и вероятностные модели
- Поведенческое и ссылочное ранжирование
- Машинное обучение ранжированию
- Рубежный контроль

Часть III: нейропоиск

- Нейропоиск I: семантический матчинг
- Нейропоиск II: переранжирование
- Нейропоиск III: векторный поиск
- Сравнение подходов
- Финальный проект

Образовательные результаты

По окончании курса вы научитесь:

- Понимать архитектуру и принципы работы современных поисковых систем.
- Применять классические модели информационного поиска (TF-IDF, BM25).
- Использовать методы машинного обучения для решения задачи ранжирования (Learning to Rank).
- Внедрять нейросетевые подходы для семантического поиска (эмбеддинги, DSSM, BERT).
- Работать с технологиями векторного поиска для нахождения семантически близких документов.
- Реализовывать ключевые компоненты поискового движка.

Рубежный контроль

Рубежный контроль

Когда: 21 октября 2026 г., в обычное время занятия
Формат: онлайн, как обычное занятие
Что проверяем: материал первой половины курса — от индексации и булева поиска до поведенческого и ссылочного ранжирования
Отдельных баллов в БРС за рубежный контроль нет, но пройти его обязательно — это условие получения итоговой оценки
Специально готовиться не нужно: достаточно разобраться в материале лекций и домашних заданий

Домашние задания

Домашние задания

Всего в курсе предусмотрено 4 домашних задания:

- Большинство заданий будут оформлены в виде соревнований на платформе Kaggle. Для успешной сдачи необходимо превзойти базовое решение (бейзлайн).
- Код решения предоставляется через Merge Request в специальном репозитории GitLab.

Задание	Баллы
ДЗ 1. Индексация и булев поиск	10
ДЗ 2. Текстовое ранжирование	10
ДЗ 3. Learning to Rank	15
ДЗ 4. Нейропоиск	15
Защита проекта	50

Итого 100 баллов
к каждому ДЗ будут критерии оценивания :)

Проверка домашних заданий

Разбалловка по курсу

Итоговая оценка (10-балльная)	Процент от максимума	Описание
10	100%	максимальный результат
9	90–99%	отлично
8	80–89%	очень хорошо
7	70–79%	хорошо
6	60–69%	хорошо
5	50–59%	удовлетворительно
4	40–49%	минимальный порог
3	30–39%	неудовлетворительно
2	20–29%	явно неудовлетворительно
1	10–19%	ничего не сделано

Кому писать с вопросами и обратной связью

Если что-то пошло не так...

Комьюнити-менеджер

Саппорт info@education.vk.company

Преподаватели курса в общем чате

Обратная связь

– К каждому занятию на платформе
– В чате с преподавателями
– В конце курса будет обязательная форма обратной связи.

Как дойти до конца курса и получить максимум пользы

Идем до конца

– Четко сформулируйте цель - зачем вы пришли на курс?

– Учебные кризисы - это нормально.

– Тайм-менеджмент наше все. Заносите в календарь лекции и семинары, время на выполнение домашнего задания и РК.

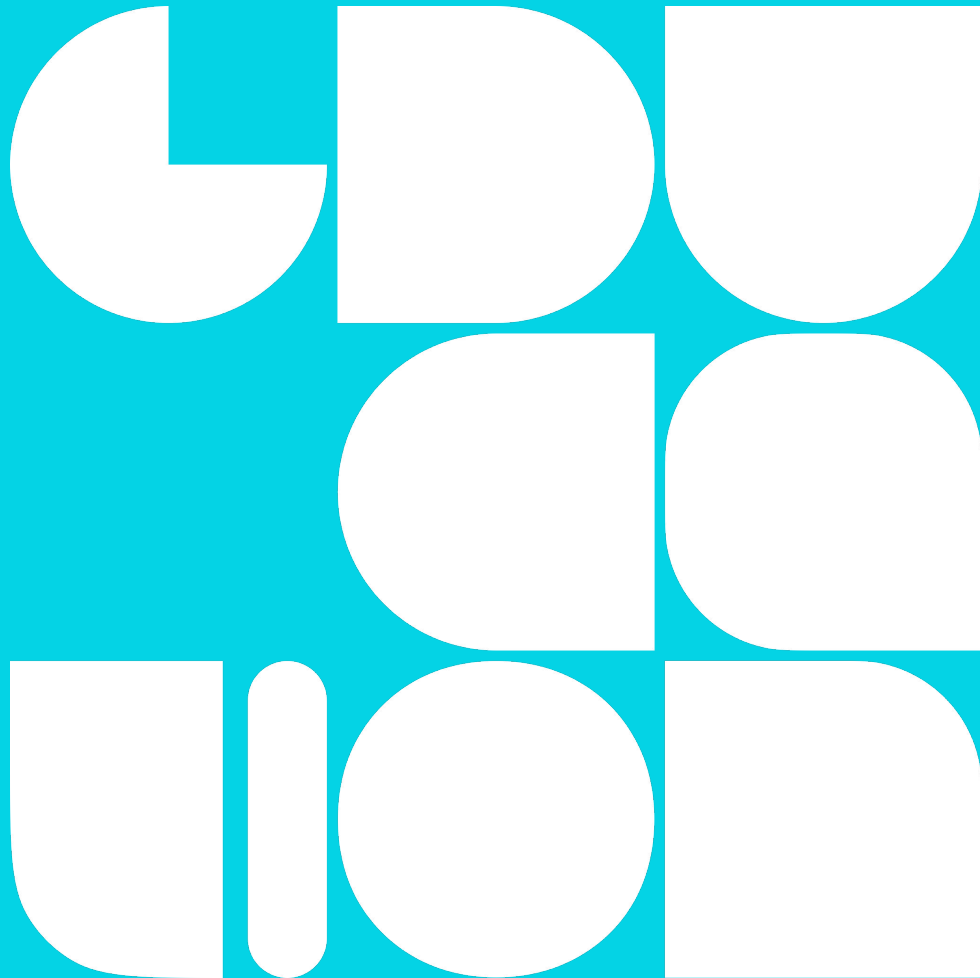
– Интересуйтесь и не стесняйтесь задавать вопросы

Напутствие от экспертов курса

Информационный поиск — это крайне интересная и динамичная область в IT.
Знания, которые вы здесь получите, находятся на стыке инженерии, машинного обучения и анализа данных.
Не бойтесь экспериментировать, задавать вопросы и глубоко погружаться в темы.

Успехов в обучении!

Что такое инфопоиск





Информацио́нный по́иск (англ. *information retrieval*) — процесс поиска *неструктурированной* документальной информации, удовлетворяющей информационные потребности, и наука об этом поиске.

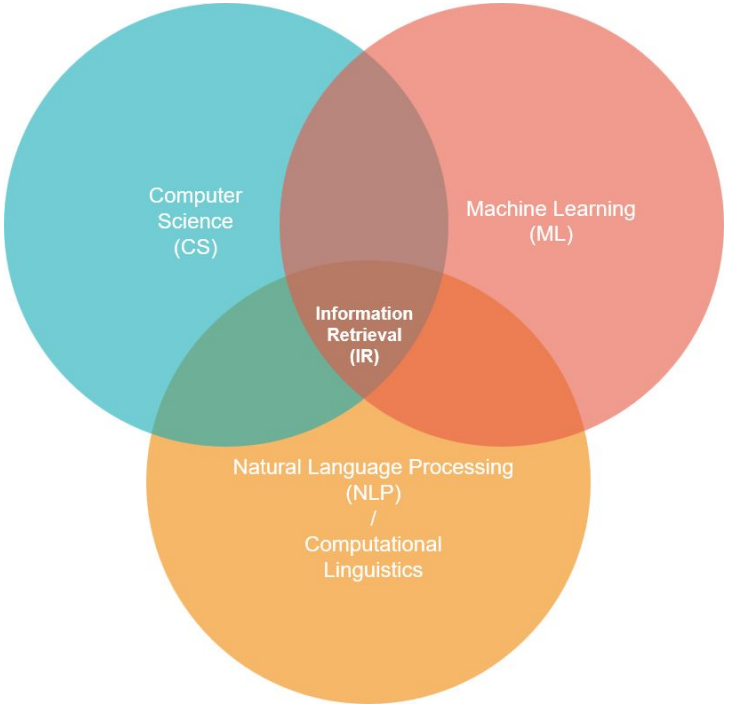
Что тут важно:

- наука
- неструктурированная информация

Википедия:
https://ru.wikipedia.org/wiki/Информационный_поиск

Information Retrieval (IR) как наука

Междисциплинарная наука где-то на стыке CS, ML и NLP
Каждый день выходит 5–15 статей в cs.IR: arxiv.org/list/cs.IR/recent
Крупные ежегодные конференции: SIGIR, CIKM, WSDM, ECIR



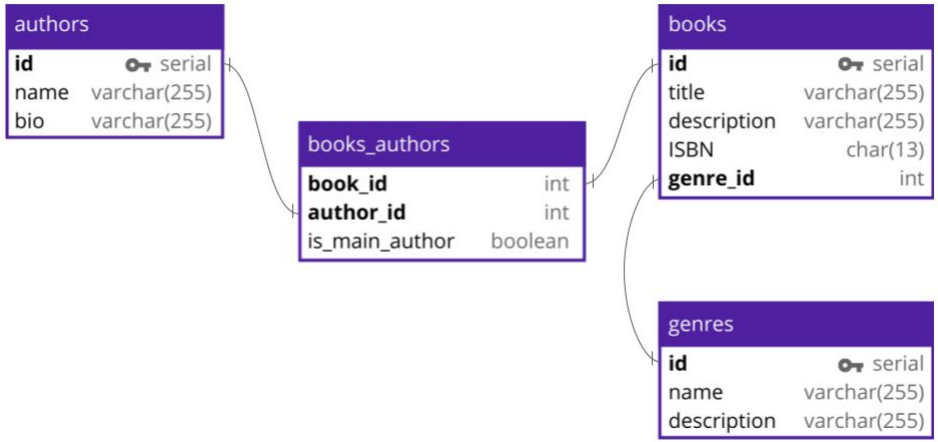
Еще IR пересекается с:

- Computer Vision (CV)
- Information Science
- Library Science

Структурированная информация

Реляционные базы данных (RDBMS) – это НЕ инфопоиск!

- Информация хранится в таблицах в структурированном виде
- Поддерживают запросы на структурированном языке запросов (SQL)
- В нашем курсе мы НЕ будем рассматривать **RDBMS**



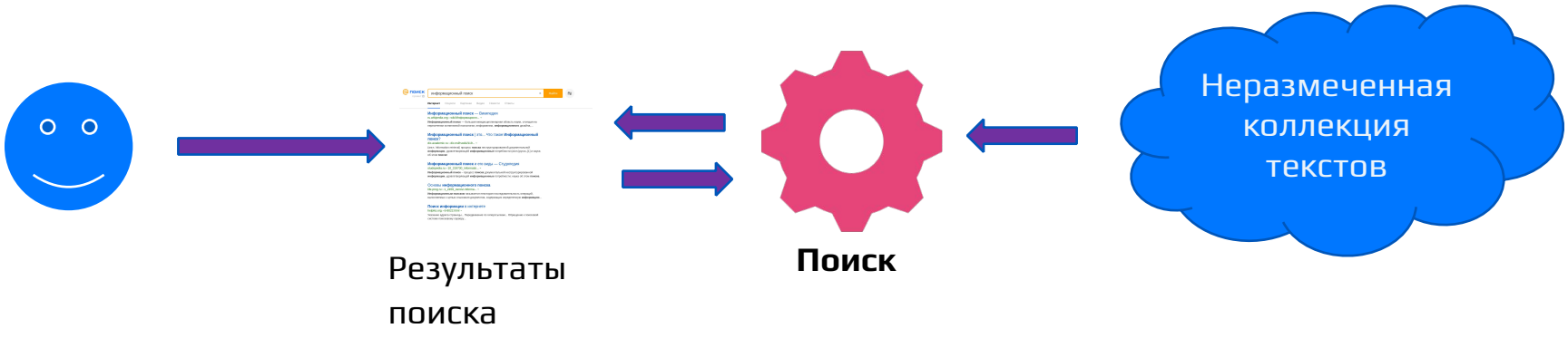
* многие современные RDBMS (напр. PostgreSQL, MySQL) поддерживают также и полнотекстовый поиск

Неструктурированная информация

Неструктурированная информация – это когда:

- Ищем по «сырым» данным – неразмеченным текстам (но в последнее время также изображениям, видео- или аудио-файлам)
- Пользователь подает запросы в максимально свободной форме

Иногда говорят про «полуструктурированную» (semi-structured) информацию если на вход системе подается текст в форматах HTML, XML, электронных документов (PDF, DOC), ...

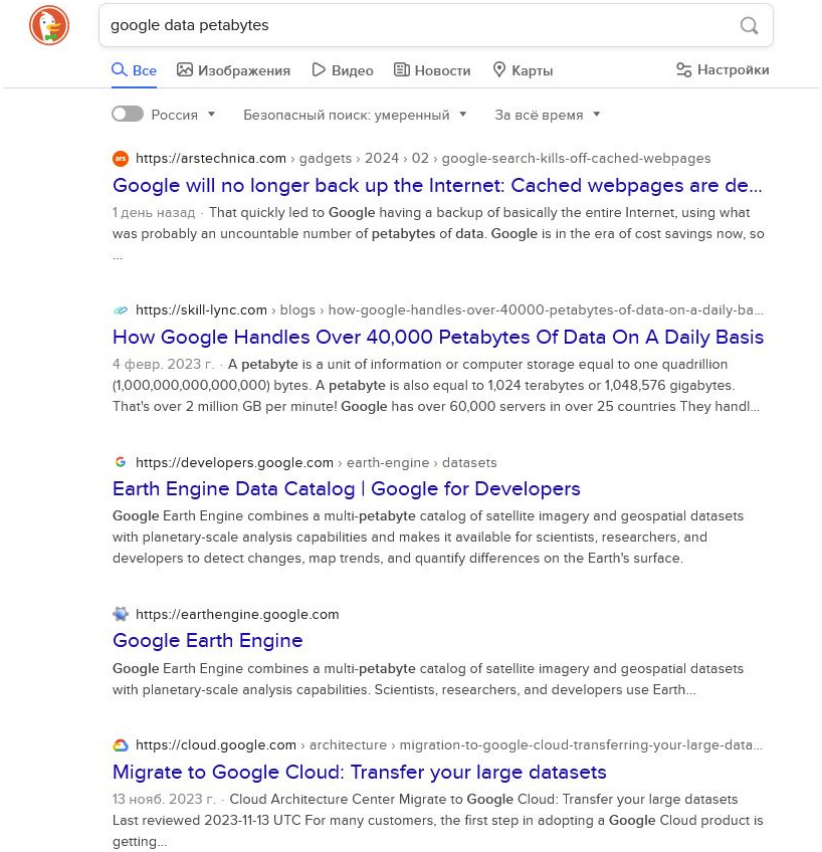


Какие бывают поиски?

Веб-поиск

Примеры: Google, Bing, Baidu, Yandex, DuckDuckGo*

- Огромные объемы данных: > 100 PB
- Огромная нагрузка: >10^8 запросов/день
- Очень дорого: >10^5 серверов
- Очень сложно: >10^3 разработчиков/админов

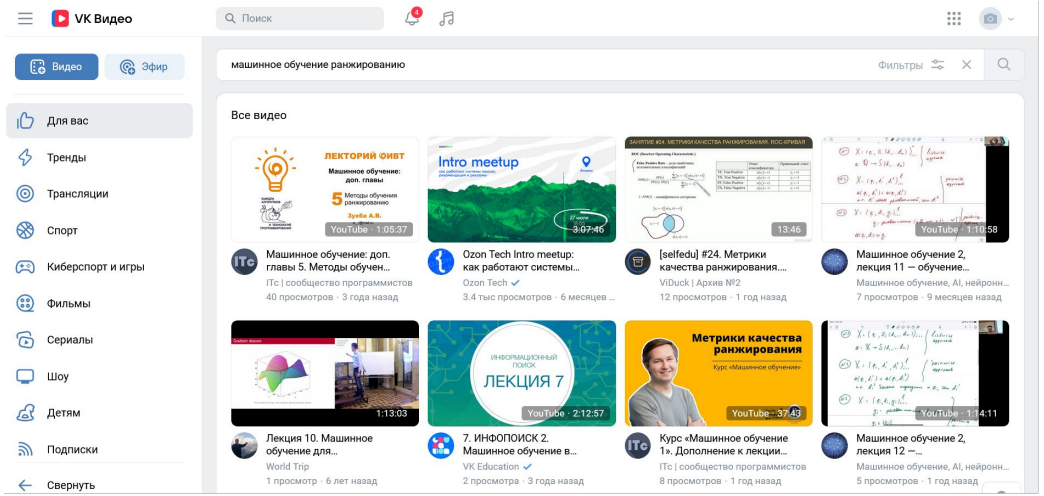


*DuckDuckGo – пример т.н. федеративного поиска (*federated search*) с использованием ресурсов других поисковиков

Поиск на сайте

Масштаб – от небольшого интернет-магазина до поиска по:

- крупнейшим соцсетям: ВКонтакте, Одноклассники, соцсеть-которую-нельзя-называть, ...
- маркетплейсам: Ozon, Wildberries, ...
- сайтам объявлений: Авито, hh.ru, Циан, ...
- поиск по Википедии
- ... и много другое



Пример: поиск по VK Видео (vkvideo.ru)
один из наших самых высоконагруженных проектов
> 1 млрд видео в базе
DAU > 4 млн
> 20 млн запросов в день

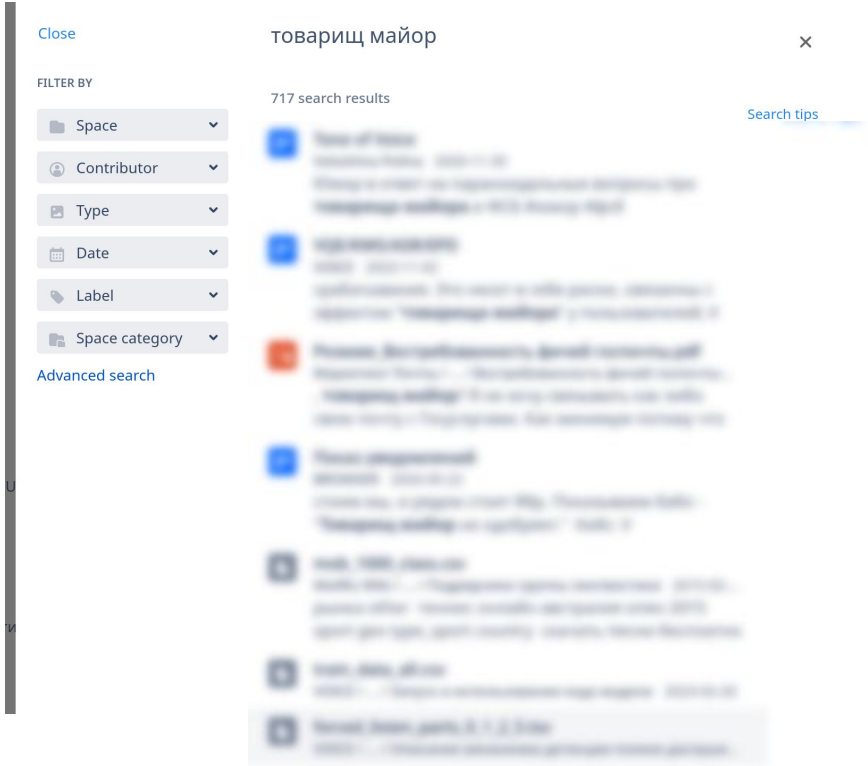
Корпоративный поиск

Поиск по:

- корпоративным базам данных
- внутренней документации

До 90-х годов почти весь поиск был корпоративным

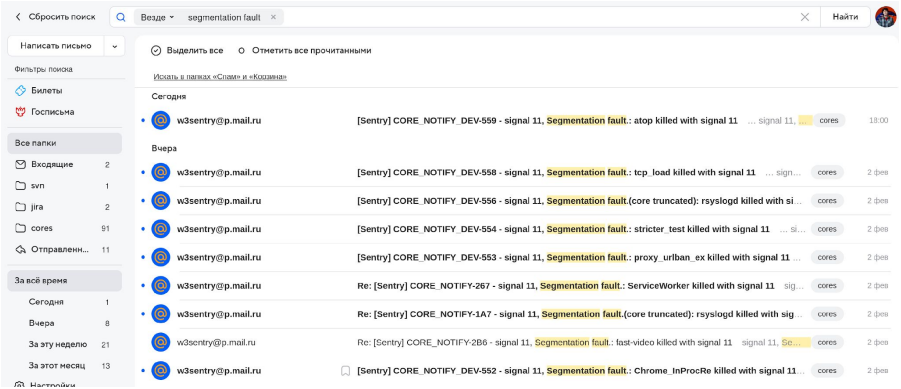
Поиск по внутренней документации компании VK в Atlassian Confluence



Персональный поиск

- Десктопный (локальный) поиск
- Поиск внутри приложения (напр. почты или мессенджера)
- Относительно небольшие объемы данных
- Часто приходится искать по приватным данным

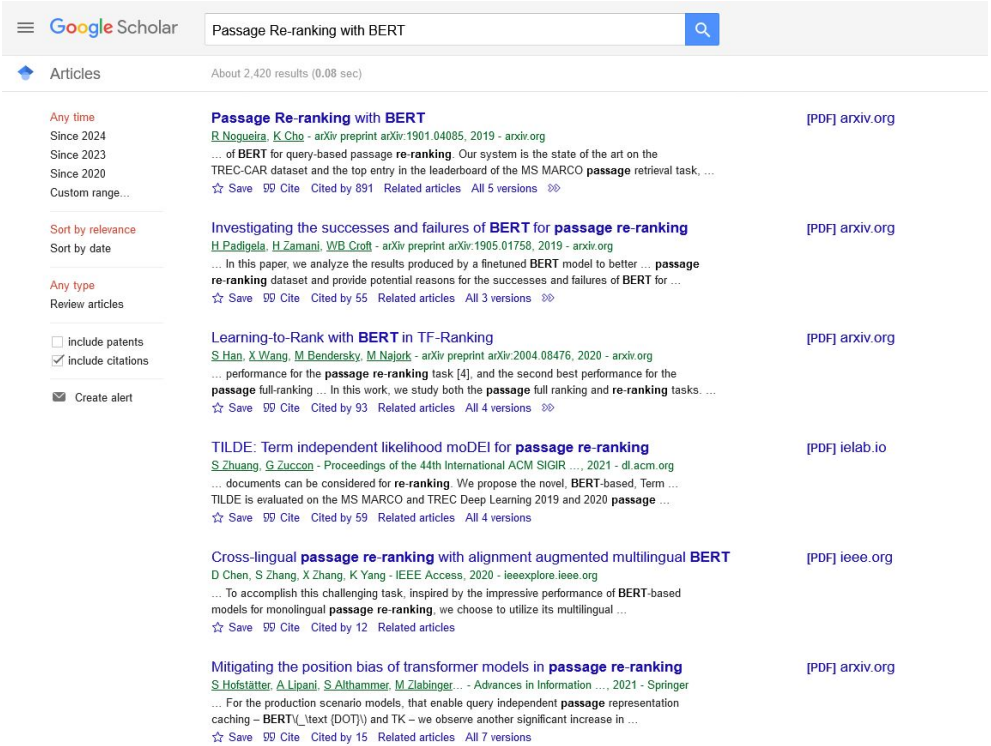
Поиск по почте mail.ru



Вертикальный поиск

Поиск по какой-то конкретной «вертикали» – контенту определенной тематики, например:

- научные статьи: https://scholar.google.com/
- вакансии и резюме: https://www.indeed.com/
- объявления
- новости
- … и т.п.



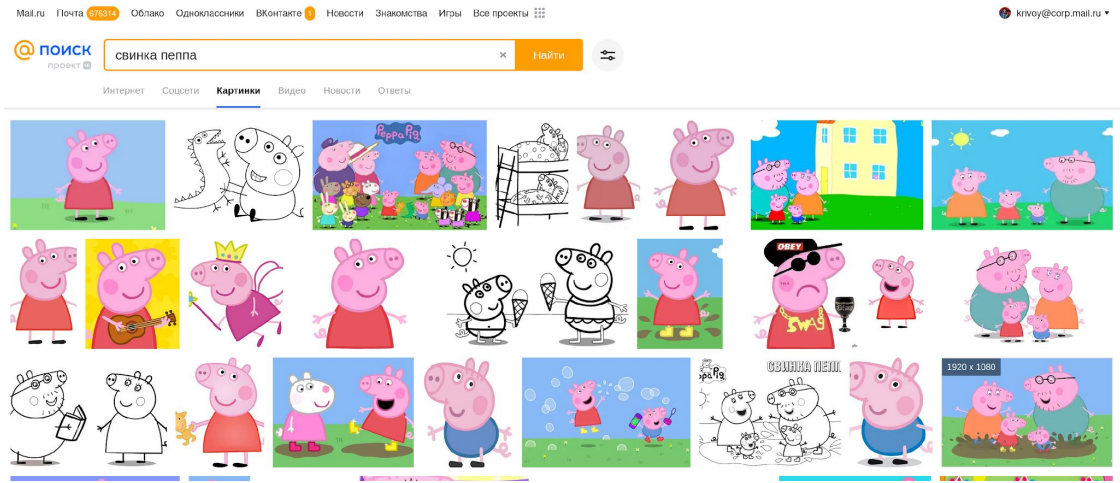
Поиск по статьям Google Scholar (https://scholar.google.com/)

Мультимедийный поиск

Мультимедийный, мультимодальный и кросс-модальный поиск по:

- изображениям
- видео
- аудио

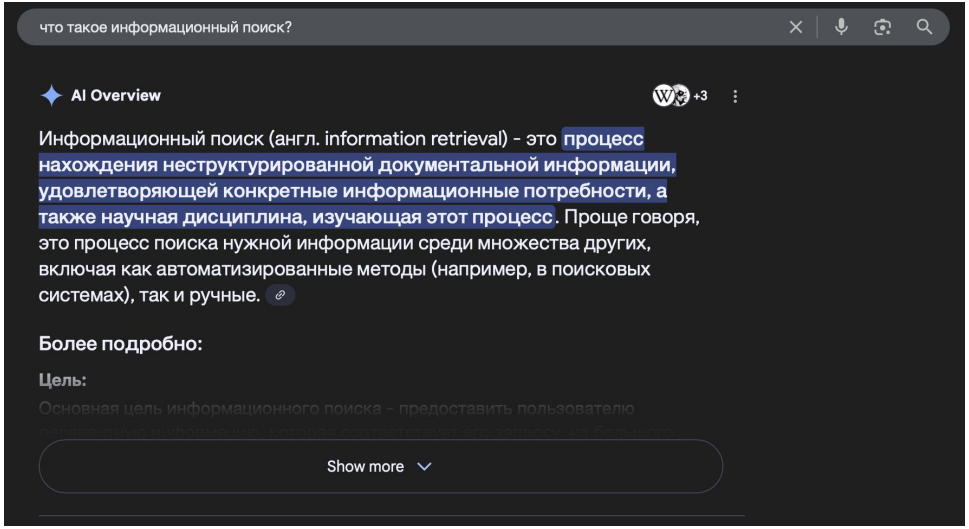
Такой поиск тоже относят к вертикальным, напр. говорят о «вертикали картинок»



Поиск по картинкам для Маруси

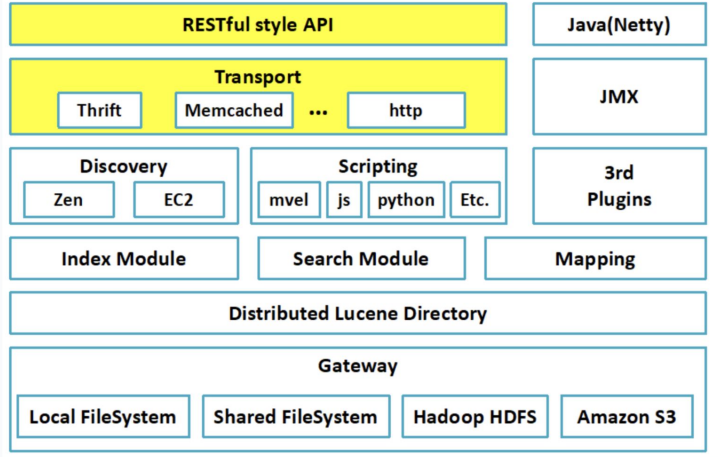
Что насчет AI?

- Чат-боты, вопросно-ответные системы (Question Answering, QA), персональные ассистенты на базе ИИ – тоже не совсем инфопоиск (но тесно с ним связаны!)
- В этом курсе будем часто про них вспоминать



Поисковик vs поисковый движок

- **Поисковик** (или поисковая система) – это поиск «в целом», напр. Google или Yandex
- **Поисковый движок** – это код, который работает «под капотом» у поисковика
- Поисковый движок могут разрабатывать другие люди, напр. в формате open source проекта
- Самый популярный поисковый движок – **Elasticsearch**
- В английском search engine обычно обозначает и то, и то
- Есть 2 способа сделать поисковик...

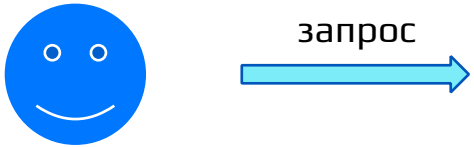


Внутренняя архитектура Elasticsearch – самого популярного на сегодня поискового движка

Простой способ сделать поисковик

1. Берем поисковый движок с открытым кодом, напр. **Lucene** или **ElasticSearch**
2. Допиливаем его под свою инфраструктуру
3. Profit!

Отличное решение для небольшого* проекта!



Поисковик на базе готового поискового движка



*небольшой – понятие относительное, напр. Ozon использует под капотом lucene: Как мы делали свой поиск в Ozon: эволюция архитектуры от SQL до O2

Сложный способ сделать поисковик

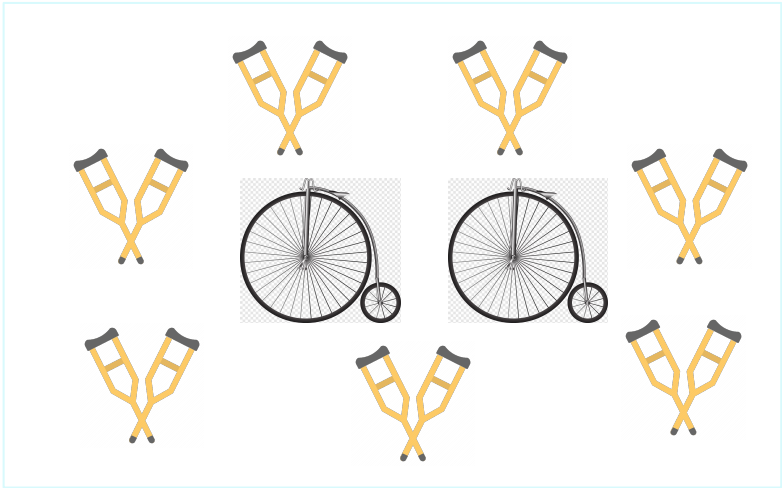
Компания полностью разрабатывает поисковик «с нуля»:

- Подойдет только *очень* большим проектам
- Дает максимальную гибкость
- Так делают Google, Yandex и VK
- Не пытайтесь повторить это дома!



запрос

Полностью самописный поисковик



Примеры движков

- Экосистема Lucene: сам **Lucene**, **Elasticsearch**, **Solr**, **OpenSearch** (написаны на Java, популярны в корпоративном мире)
- **Sphinx** и его потомок **Manticore** (написаны на C++)
- «Академические» движки: **anserini**, **terrier**, «лемуры» (**indri**, **galago**) – удобны для исследователей, но не подходят для использования «в бою»
- Во многих RDBMS встроены движки полнотекстовго поиска, напр. в **PostgreSQL** – удобно если вы уже используете RDBMS
- Search-as-a-service: **Algolia**

☐ include secondary database models — 26 systems in ranking, February 2024

Rank			DBMS	Database Model	Score		
Feb 2024	Jan 2024	Feb 2023			Feb 2024	Jan 2024	Feb 2023
1.	1.	1.	Elasticsearch	Search engine, Multi-model	135.74	-0.33	-2.86
2.	2.	2.	Splunk	Search engine	91.65	-1.07	+4.57
3.	3.	3.	Solr	Search engine, Multi-model	44.60	-0.53	-1.28
4.	4.	4.	OpenSearch	Search engine, Multi-model	14.84	+0.88	+4.34
5.	5.	5.	MarkLogic	Multi-model	7.49	-0.29	-1.35
6.	6.	↑ 7.	Algolia	Search engine	6.40	-0.09	-0.62
7.	↑ 8.	↑ 8.	Sphinx	Search engine	6.08	-0.01	-0.55
8.	↓ 7.	↓ 6.	Microsoft Azure AI Search	Search engine, Multi-model	6.00	-0.22	-1.13
9.	9.	9.	Virtuoso	Multi-model	5.31	+0.23	-0.79
10.	10.	10.	ArangoDB	Multi-model	4.34	-0.01	-0.95
11.	11.	↑ 12.	Coveo	Search engine	2.64	+0.12	+0.27
12.	12.	↓ 11.	Amazon CloudSearch	Search engine	2.33	-0.10	-0.13
13.	13.	↑ 14.	Meilisearch	Search engine	1.44	-0.01	+0.35
14.	↑ 15.	↑ 18.	Typesense	Search engine	1.03	+0.00	+0.61
15.	↓ 14.	↓ 13.	Xapian	Search engine	1.02	-0.21	-0.17
16.	16.	↓ 15.	CrateDB	Multi-model	0.91	-0.09	0.00
17.	17.	↓ 16.	SearchBlox	Search engine	0.70	-0.13	-0.02
18.	18.		Vespa	Multi-model	0.60	-0.06	
19.	↑ 20.	19.	Alibaba Cloud Log Service	Search engine	0.42	-0.01	+0.11
20.	↓ 19.	↓ 17.	Marqo	Search engine	0.40	-0.07	-0.05
21.	21.	21.	Manticore Search	Search engine, Multi-model	0.25	-0.09	-0.06
22.	↑ 23.	22.	Tigris	Multi-model	0.09	-0.13	-0.20
23.	↑ 24.	↑ 24.	Indica	Search engine	0.08	-0.01	+0.08
24.	↓ 22.	↓ 20.	Exorbyte	Search engine	0.07	-0.19	-0.24

Топ поисковых движков по популярности на 2024 г.

(https://db-engines.com/en/ranking/search+engine)

whoosh

Мы будем активно использовать движок **whoosh**:

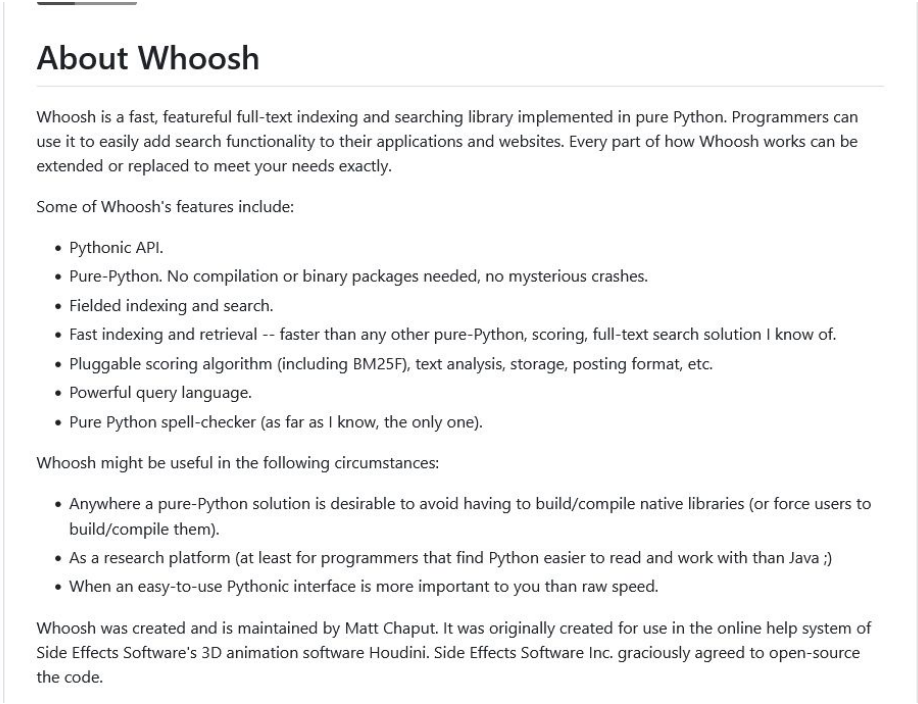
- Репа: https://github.com/mchaput/whoosh
- Документация: https://whoosh.readthedocs.io/en/latest/

Плюсы:

- Pure python
- Нет внешних зависимостей
- Очень простой в использовании

Главный минус:

- Не production ready: **медленный** и местами глючный (но нас это не пугает)



About Whoosh

Whoosh is a fast, featureful full-text indexing and searching library implemented in pure Python. Programmers can use it to easily add search functionality to their applications and websites. Every part of how Whoosh works can be extended or replaced to meet your needs exactly.

Some of Whoosh's features include:

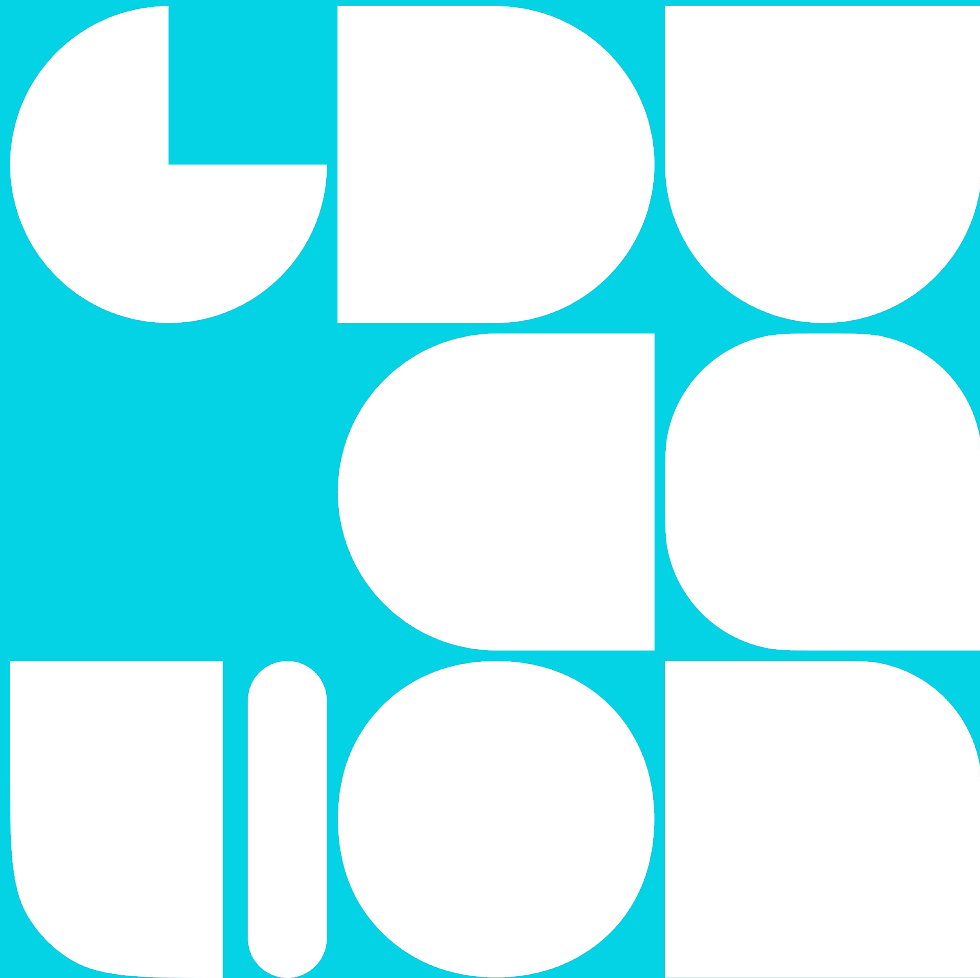
- Pythonic API.
- Pure-Python. No compilation or binary packages needed, no mysterious crashes.
- Fielded indexing and search.
- Fast indexing and retrieval -- faster than any other pure-Python, scoring, full-text search solution I know of.
- Pluggable scoring algorithm (including BM25F), text analysis, storage, posting format, etc.
- Powerful query language.
- Pure Python spell-checker (as far as I know, the only one).

Whoosh might be useful in the following circumstances:

- Anywhere a pure-Python solution is desirable to avoid having to build/compile native libraries (or force users to build/compile them).
- As a research platform (at least for programmers that find Python easier to read and work with than Java ;)
- When an easy-to-use Pythonic interface is more important to you than raw speed.

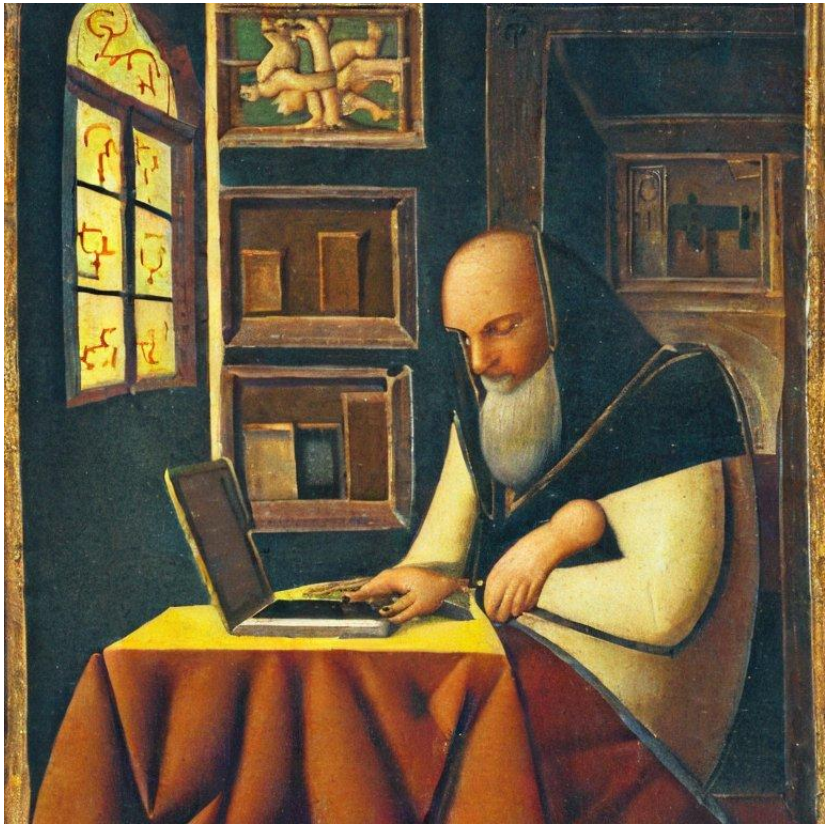
Whoosh was created and is maintained by Matt Chaput. It was originally created for use in the online help system of Side Effects Software's 3D animation software Houdini. Side Effects Software Inc. graciously agreed to open-source the code.

Эволюция инфопоиска



Немного истории

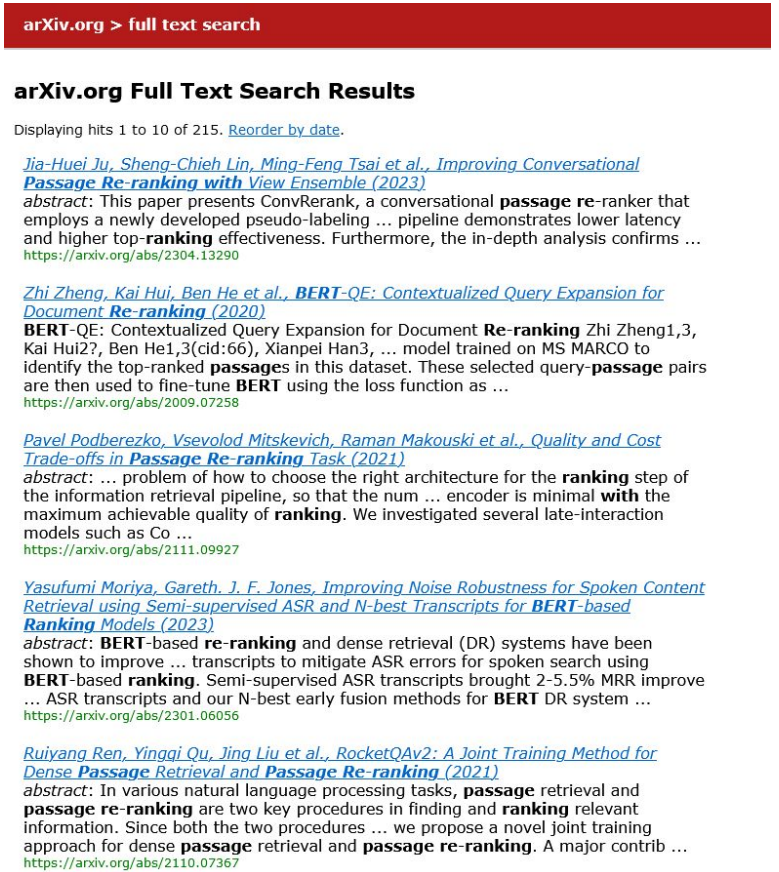
- Лучший способ понять что-либо – понять как это что-то появилось и развивалось
- Делаем упор не на даты а на эволюцию идей
- Начнем с **grep** и постепенно придем к современному поисковику



Информационный поиск в Средневековье глазами Stable Diffusion 2.1

Постановка задачи

- Дана маленькая коллекция (корпус) из 1 млн. текстовых документов
- Храним документы в виде файлов на жестком диске (HDD)
- Пользователи подают запросы в текстовом виде
- Задача: эффективно находить документы которые содержат ключевые слова запроса
- Такие задачи возникли уже в 50-е годы в рамках задач по автоматической обработке научной литературы



arXiv.org > full text search

arXiv.org Full Text Search Results

Displaying hits 1 to 10 of 215. Reorder by date.

*Jia-Huei Ju, Sheng-Chieh Lin, Ming-Feng Tsai et al., Improving Conversational **Passage Re-ranking with** View Ensemble (2023)*
abstract: This paper presents ConvRerank, a conversational **passage re**-ranker that employs a newly developed pseudo-labeling ... pipeline demonstrates lower latency and higher top-**ranking** effectiveness. Furthermore, the in-depth analysis confirms ...
https://arxiv.org/abs/2304.13290

*Zhi Zheng, Kai Hui, Ben He et al., **BERT**-QE: Contextualized Query Expansion for Document **Re-ranking** (2020)*
BERT-QE: Contextualized Query Expansion for Document **Re**-**ranking** Zhi Zheng1,3, Kai Hui2?, Ben He1,3(cid:66), Xianpei Han3, ... model trained on MS MARCO to identify the top-ranked **passage**s in this dataset. These selected query-**passage** pairs are then used to fine-tune **BERT** using the loss function as ...
https://arxiv.org/abs/2009.07258

*Pavel Podberezko, Vsevolod Mitskevich, Raman Makouski et al., Quality and Cost Trade-offs in **Passage Re-ranking** Task (2021)*
abstract: ... problem of how to choose the right architecture for the **ranking** step of the information retrieval pipeline, so that the num ... encoder is minimal **with** the maximum achievable quality of **ranking**. We investigated several late-interaction models such as Co ...
https://arxiv.org/abs/2111.09927

*Yasufumi Moriya, Gareth. J. F. Jones, Improving Noise Robustness for Spoken Content Retrieval using Semi-supervised ASR and N-best Transcripts for **BERT**-based **Ranking** Models (2023)*
abstract: **BERT**-based **re**-**ranking** and dense retrieval (DR) systems have been shown to improve ... transcripts to mitigate ASR errors for spoken search using **BERT**-based **ranking**. Semi-supervised ASR transcripts brought 2-5.5% MRR improve ... ASR transcripts and our N-best early fusion methods for **BERT** DR system ...
https://arxiv.org/abs/2301.06056

*Ruiyang Ren, Yingqi Qu, Jing Liu et al., RocketQAv2: A Joint Training Method for Dense **Passage** Retrieval and **Passage Re-ranking** (2021)*
abstract: In various natural language processing tasks, **passage** retrieval and **passage re**-**ranking** are two key procedures in finding and **ranking** relevant information. Since both the two procedures ... we propose a novel joint training approach for dense **passage** retrieval and **passage re**-**ranking**. A major contrib ...
https://arxiv.org/abs/2110.07367

Полнотекстовый поиск по 2.4 млн научных статей на arxiv.org:

https://search.arxiv.org/?in=&query=Passage%20Re-ranking%20with%20BERT

grep

- Первое что приходит в голову
- Доступен в любой UNIX-системе
- Поддерживает регулярные выражения
- Но это плохая идея даже для относительно небольших коллекций – почему?

```
$ grep -r "ugly hack" /usr/share/

/usr/share/doc/libssh2/NEWS:     This
ugly hack makes `ld` happy and detection
succeed.
/usr/share/perl5/5.36/Archive/Tar.pm:
### this is one ugly hack =/ but needed
for direct extraction
/usr/share/perl5/5.36/Pod/Perldoc/ToMan.p
m:     # working, apparently, so there's
this ugly hack:
/usr/share/perl5/5.36/TAP/Formatter/Base.
pm:    # ugly hack.  Must rethink this :(
```

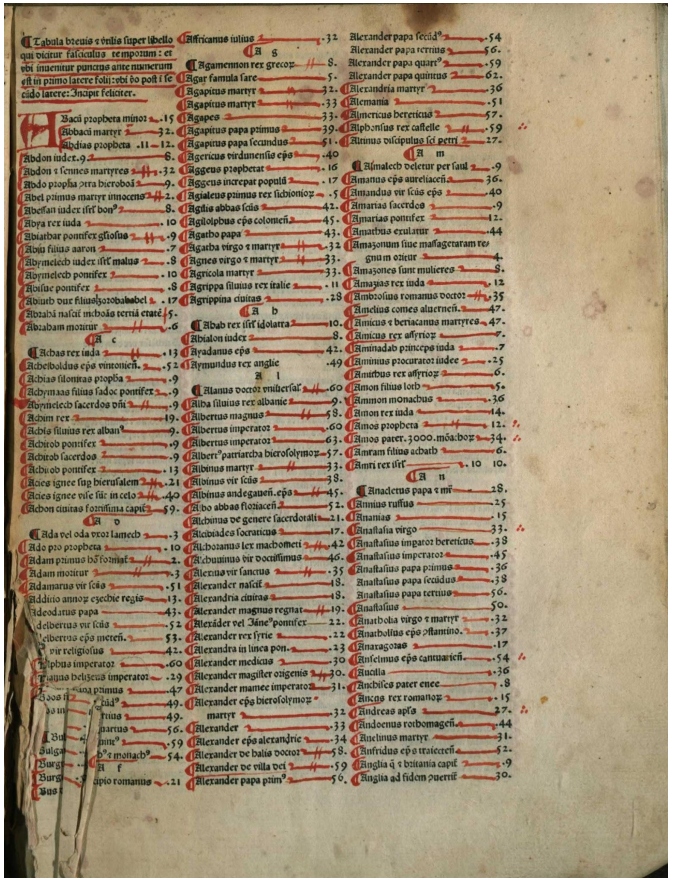
(Не) эффективность grep

Посчитаем время поиска в нашей небольшой коллекции:

- Всего 1 млн. документов
- Допустим, что типичный документ содержит 1000 слов
- 1 слово в среднем состоит из 5 букв и занимает 10 байт
- Всего нам потребуется: 10^6 * 10^3 * 10 = 10 GB
- Типичная скорость последовательного чтения с HDD: ~150 MB/s
- Наш grep займет больше 1 минуты!

Индекс: идея

- А что если мы предрассчитаем и сохраним список ключевых слов и документов, в которых они упомянуты
- Вместо линейного поиска в огромной текстовой коллекции получим быстрый поиск в небольшом списке ключевых слов
- Впервые появились в 13 в. виде *книжных указателей* (*book index*)
- Переносит тяжесть вычислений с *онлайна (*)* в *оффлайн (**)*
- Работает «под капотом» любого поискового движка



Индекс из *Fasciculus temporum* Вернера Ролевинка 1481 г. издания

* Онлайн – это все, что происходит во время обработки поискового запроса

** Оффлайн – это то, чем поисковик занимается все остальное время

Обратный (инвертированный) индекс

- На входе мини-коллекция из 3х документов:
  1. *Московский физико-технический институт*
  2. *Московский государственный университет*
  3. *Университет ИТМО*



Обратный индекс

- *государственный* → [2]
- *институт* → [1]
- *итмо* → [3]
- *московский* → [1, 2]
- *технический* → [1]
- *физика* → [1]
- *университет* → [2, 3]

Слова нормализованы!

Ссылаемся на документ через его ID.

Храним списки docID в сортированном виде.

Как реализовать обратный индекс?

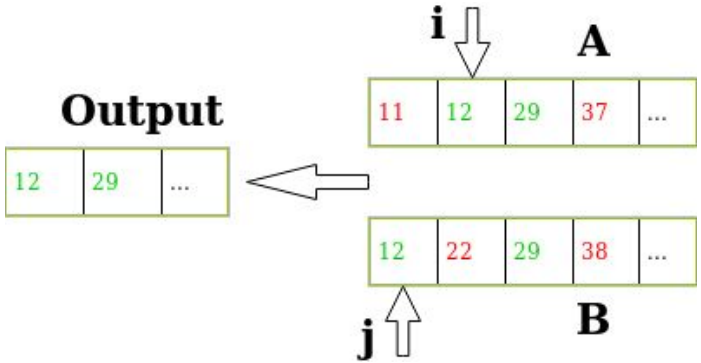
- Наивная реализация: дерево поиска или хэш-таблица, в которой хранятся списки документов
- Основная проблема: надо очень много оперативной памяти (RAM)
- На практике, часть индекса хранится в оперативной памяти, а часть на диске
- Для очень больших коллекций (> 10 млн. документов) индекс приходится нарезать на части, которые лежат на разных серверах
- Эффективное построение индекса – это тоже не просто
- Подробно разберем детали реализации индекса в лекции №2 про обратный индекс и булев поиск

Обработка многословных запросов

- Как теперь с помощью обратного индекса эффективно обрабатывать поисковые запросы?
- С однословными запросами (напр. «*университет*») все тривиально: ищем ключевое слово в индексе и выдаем все связанные с ним документы
- Как быть с многословными, напр. «*университет итмо*»?

И-запросы

- Сделаем сильное предположение!
- Многословный запрос – это просто конъюнкция нескольких слов
  - *университет итмо* → *университет && итмо*
- Такие запросы очень просто интерпретировать:
  - Документ должен содержать ВСЕ слова: И *университет*, И *итмо*
- И очень просто обрабатывать:
  - Находим связанные с каждым из слов запроса списки документов
  - **Пересекаем** эти списки с помощью простого алгоритма (см. картинку)



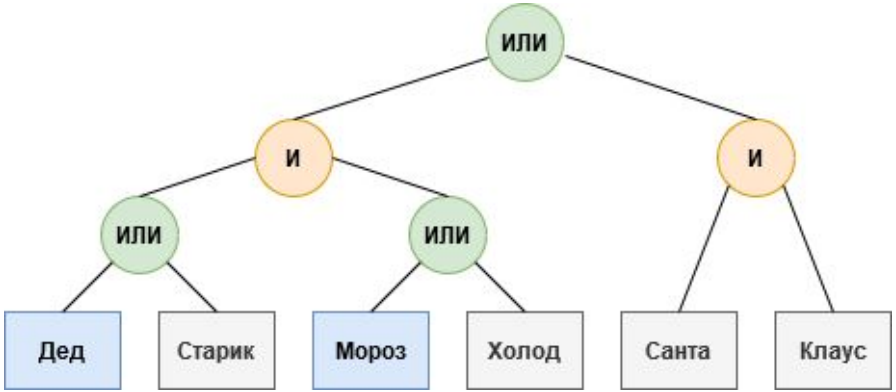
Алгоритм пересечения 2-х сортированных списков документов

ИЛИ-запросы

- Аналогично, можно ввести понятие ИЛИ-запросов
- Очень удобны если хотим искать с синонимами
- Запрос: *университет* || *институт*
- Дизъюнкцию тоже просто интерпретировать:
  - Документ должен содержать ХОТЯ БЫ ОДНО из слов: ИЛИ *университет*, ИЛИ *институт*
- И также просто обрабатывать: просто **сливаем** связанные с ключевыми словами списки документов

Булев поиск

- Приходим к модели булева поиска (*boolean retrieval model*)
- Моделируем многословные запросы с помощью логических операций
- Поддерживаем И- и ИЛИ-запросы, а также их комбинации произвольной вложенности
- Пример сложного запроса: *((дед || старик) && (мороз || холод)) || (санта && клаус))*
- В общем случае такой запрос можно представить в виде дерева: узлы-операции и листья-слова
- Сливаем и пересекаем найденные списки документов двигаясь снизу вверх по дереву



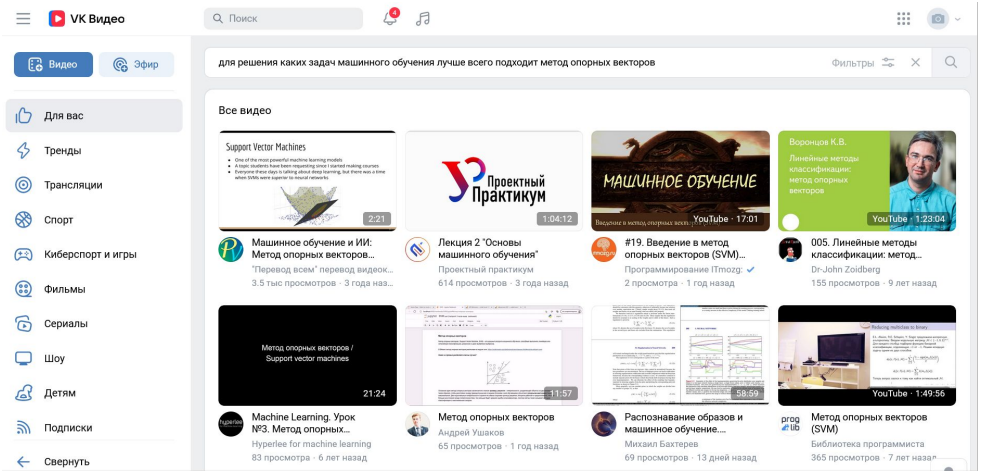
Сложный булев запрос ДЕД МОРОЗ представленный в виде дерева

Использованы синонимы:

- ДЕД МОРОЗ == САНТА КЛАУС
- ДЕД == СТАРИК
- МОРОЗ == ХОЛОД

Ad hoc* поиск

- У булева поиска есть серьезные проблемы!
- Пользователи не хотят писать сложные логические выражения
- И-запросы обрабатываются слишком жестко: мы хотим поднимать документы, в которых есть только часть слов запроса
- Пользователи не могут проглядывать глазами миллионы найденных документов, содержащих ключевые слова – нужно ранжирование!
- Нужен т.н. *ad hoc* поиск: обработка запросов в свободной форме + ранжирование результатов



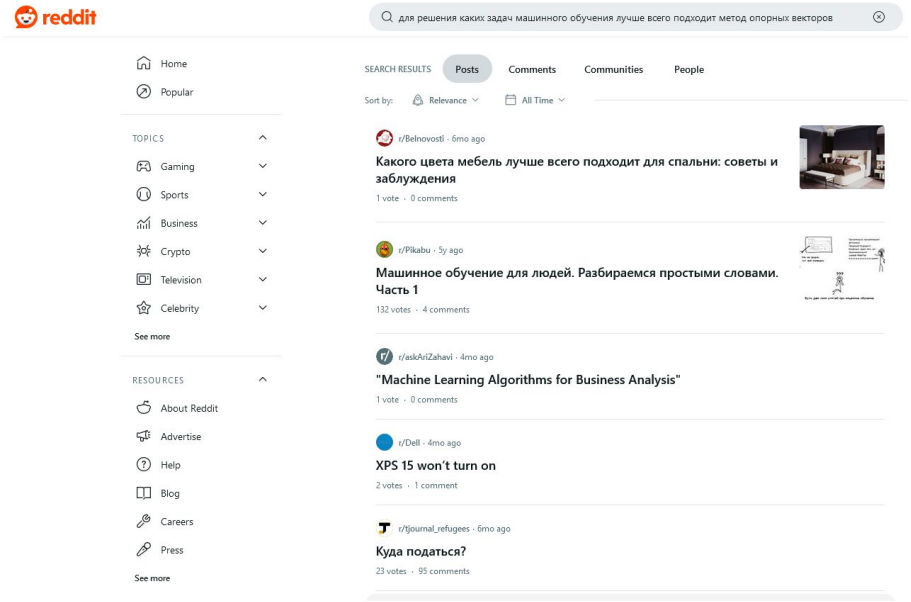
Обработка очень многословного ad hoc запроса *«для решения каких задач машинного обучения лучше всего подходит метод опорных векторов»*

Этот запрос, очевидно, нельзя обрабатывать как «строгий» И-запрос!

*__Ad hoc__ — латинская фраза, означающая «для данного случая», «специально для этого».

Задача ранжирования

- Самая важная и сложная задача информационного поиска!
- Релевантные документы должны быть выше, а нерелевантные – ниже
- *Релевантный документ* – это документ, удовлетворяющий информационную потребность пользователя
- Почти весь наш курс будет посвящен методам решения задачи ранжирования



Пример очень плохого ранжирования на reddit.com* по запросу *«для решения каких задач машинного обучения лучше всего подходит метод опорных векторов»*

Оценка релевантности

- Релевантность надо уметь измерять!
- Например, с помощью асессорской разметки
- Используются краудсорсинговые платформы, напр. Яндекс Толока
- У нас в VK есть своя внутренняя платформа (SQ, расшифровывается как Search Quality)
- В современное мире людей-асессоров все чаще и чаще заменяют LLM!

насекомые синий трактор ВСЕ ЗАПРОСЫ

ВЫБОРКА НА GOGO.ST

[youtube new] sm6, best vk model #70

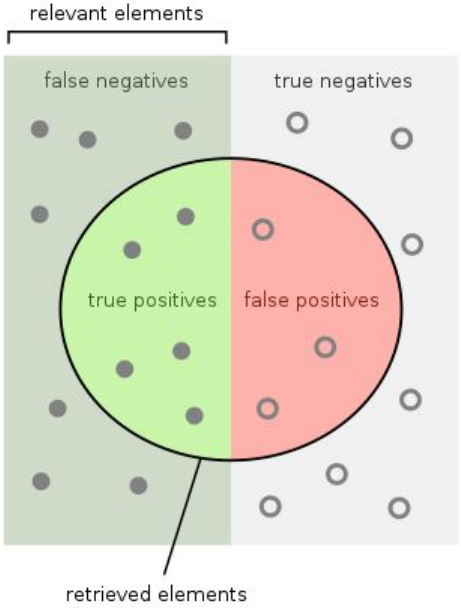
V-LDCG-10 = 19.78

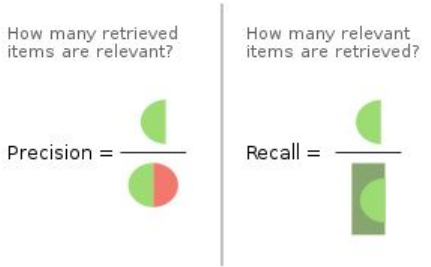
Позиция	Серп	Оценка
1	https://vk.com/video148631493_456239808	2
2	https://vk.com/video-211391900_456239201	3
3	https://vk.com/video-85859225_456240469	3
4	https://vk.com/video-7966621_456239049	1
5	https://vk.com/video217045772_456239072	2
6	https://vk.com/video-221593574_456239210	2
7	https://vk.com/video1218682_456240188	3
8	https://vk.com/video536711804_456239047	2
9	https://vk.com/video59161431_456239984	2
10	https://vk.com/video176207939_456239071	3

Пример асессорской разметки. Используется 3-х балльная шкала релевантности: 1 (нерелевантно), 2 (полезно) и 3 (релевантно)

Метрики качества поиска

- Имея оценки релевантности можно посчитать метрики качества
- Самые простые метрики: *точность* и *полнота*
- Точность – насколько релевантна та выдача, которую мы показали
- Полнота – насколько хорошо мы находим релевантные документы, которые есть в базе





Картинка из википедии (https://en.wikipedia.org/wiki/Precision_and_recall)

Текстовое ранжирование

- Как сделать хорошее ранжирование?
- Простейший способ ранжирования – используем тексты поискового запроса и документа
- Идея: чем больше слов запроса нашлось в документе – тем лучше
- Проблема: многословные запросы вида *«университет итмо»*
- Не все слова запроса одинаково важны!
- Документы в которых нашлось только слово ИТМО более полезны чем документы в которых нашлось только слово УНИВЕРСИТЕТ

TF-IDF и BM25

- Надо как-то учитывать важность слов
- Первая попытка: *TF-IDF* (70-е годы)
- Выводим важность слов из вероятностных соображений: *BM25* (80-е годы)
- Для каждой пары запрос-документ получаем число (score, напр. *TfIdfScore(Q,D)* или *Bm25Score(Q,D)*) по которому можно ранжировать документы!
- Можно пойти дальше и учесть дополнительные факторы: близость слов, зону в которой нашлись ключевые слова (напр. Title или Body) и т.п.
- Все это и многое другое подробно разберем в лекциях №4 и №5 про текстовое ранжирование

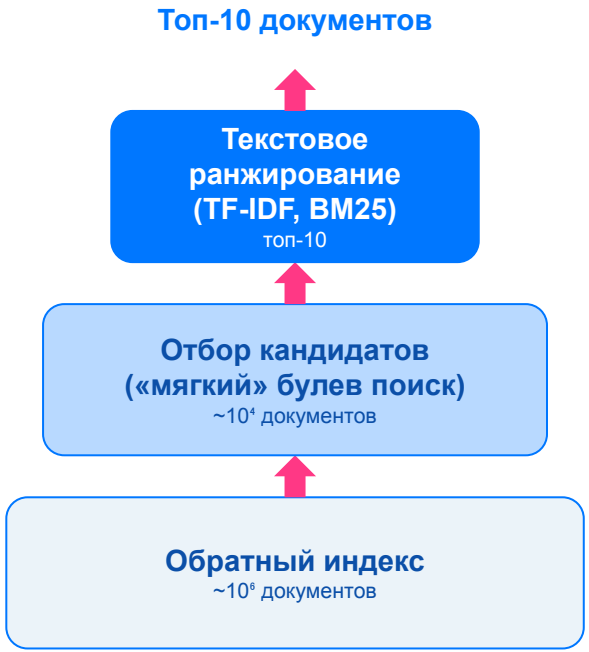
Given a query Q, containing keywords $q_1, \ldots, q_n$, the BM25 score of a document D is:

$$\text{score}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

$f(q_i, D)$ — сколько раз слово запроса встретилось в документе
$|D|$ — длина документа, avgdl — средняя длина по коллекции
k_1 и b — гиперпараметры, обычно $k_1 \approx 1.2–2.0$ и $b \approx 0.75$
$\text{IDF}(q_i)$ — насколько слово редкое, а значит важное
Разберём формулу и её вывод в лекции №5

Поиск на начало 90-х годов

- Ранжирование состоит из 2-х этапов
- Этап *отбора кандидатов*: ищем документы которые содержат ключевые слова
- Отбор кандидатов работает по принципу булева поиска но с «мягким» И – не требуем обязательного наличия всех слов
- Этап текстового ранжирования: ранжируем отобранных ранее кандидатов с помощью простой формулы, напр. TF-IDF или BM25
- Именно так работает поисковый движок **whoosh**



Революция №1: веб-поиск

- В начале 90-х годов появился web (не путать с Интернет!)
- Сначала появились веб-серверы и браузеры (напр. Mosaic, 1993)
- Потом появились ручные каталоги (1993)
- И, наконец, появились первые поисковики: Lycos (1994), AltaVista (1995), Yahoo! Search (1995) и т.п.
- Размеры индекса растут с миллионов до миллиардов документов и дальше
- Поисковики превращаются в огромные распределенные системы
- Происходит коммерциализация: появляется интернет-реклама, сначала просто баннеры, потом *контекстная реклама*



Altavista в 1996 г.

Поведенческая статистика

- Поисковики логируют действия пользователей: показы, просмотры, клики
- Из логов считаем статистики популярности документа
- Абсолютные — например, полное число кликов в документ
- Относительные, позапросные — например CTR
- Чем выше CTR документа по запросу, тем он релевантнее

$$\mathrm{CTR}(Q, D) = \frac{\text{число кликов в документ D по запросу Q}}{\text{число показов документа D по запросу Q}}$$

Значит, CTR можно использовать как score, по которому ранжируем документы

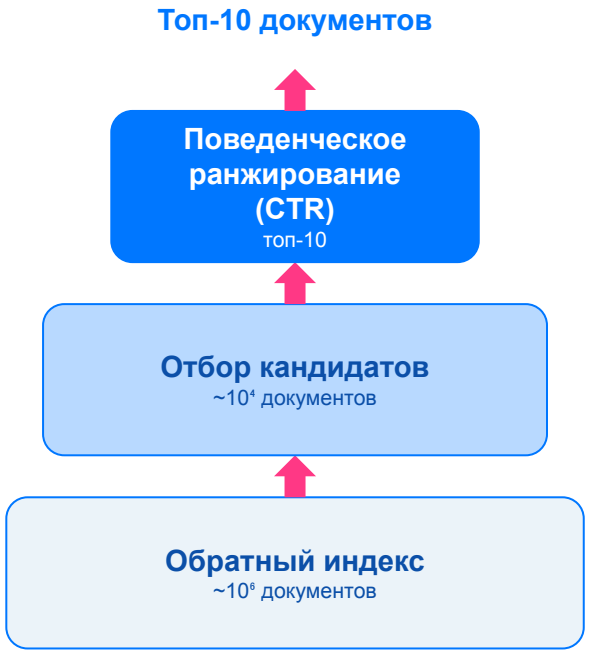
25.04.2022 14:54	url [0]	video
	url [1]	fast_social
	url [2]	http://detskie-multiki.ru/252-schenyachiy-patrul.html
	url [3]	http://schenyachiypatrul.ru/vse-serii-podryad/
	url [4]	http://mashaimedved.net/schenyachiy-patrul/
	url [5]	market
	url [6]	http://okko.tv/serial/paw-patrol-513522
	url [7]	http://vseigru.net/igry-shchenyachij-patrul.html
	url [8]	http://wiki2.org/ru/%D0%A9%D0%B5%D0%BD%D1%8F%D1%87%D0%B8%D0%B9_%D0%BF%D0%B0%D1%82%D1%80%D1%83%D0%BB%D1%8C
	url [9]	http://www.kinopoisk.ru/series/790343/
	url [10]	http://kg-portal.ru/movies/schenjachijpatrulkino/
	url [11]	http://vk.com/official_pawpatrol
	url [12]	http://fanfics.me/fandom1280/heroes
	click [0]	1
	userId	0C63622657670A39D5FC698D74B0
	timestamp	1650887694000
	click_time [0]	121
	sessionInfo	paramName [0] RID

Timestamp: 25.04.2022
Size: 657 bytes

Пример типичной залогированной поисковой сессии по запросу «щенячий патруль»

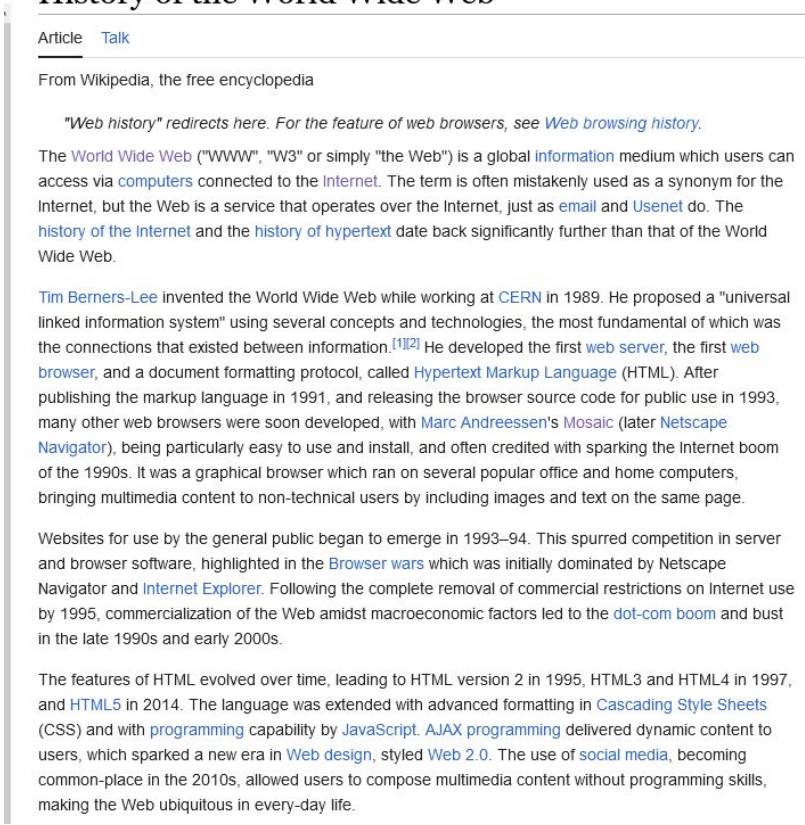
Поведенческое ранжирование

- Просто ранжируем кандидатов по CTR
- Проблема №1: т.н. *bias* – люди кликают в те документы, которые мы уже показываем (и чем выше показываем, тем больше кликают!)
- Проблема №2: *холодный старт* – мы не покажем документ, по которому еще нет статистики
- Хорошо работает только для высокочастотных запросов, напр. «*одноклассники*» в веб-поиске
- Про все это поговорим подробнее в лекции №6 про поведенческое и ссылочное ранжирование



Гиперссылки

- Web-страницы ссылаются друг на друга с помощью *гиперссылок*
- Сама идея появилась еще в 40-е годы; термин *гипертекст* впервые был использован в 1963 г.
- В 80-е годы идеи были развиты Тимом Бернерс-Ли (CERN) и стали основной мотивацией для создания web-технологий
- Люди (наивно) представляли себе web как огромную распределенную и связанную гиперссылками базу знаний
- Реальность, правда, оказалось немного другой: в 2024 г. веб – это огромные замкнутые экосистемы, такие как соцсети



History of the World Wide Web

Article Talk

From Wikipedia, the free encyclopedia

"Web history" redirects here. For the feature of web browsers, see Web browsing history.

The World Wide Web ("WWW", "W3" or simply "the Web") is a global information medium which users can access via computers connected to the Internet. The term is often mistakenly used as a synonym for the Internet, but the Web is a service that operates over the Internet, just as email and Usenet do. The history of the Internet and the history of hypertext date back significantly further than that of the World Wide Web.

Tim Berners-Lee invented the World Wide Web while working at CERN in 1989. He proposed a "universal linked information system" using several concepts and technologies, the most fundamental of which was the connections that existed between information.[1][2] He developed the first web server, the first web browser, and a document formatting protocol, called Hypertext Markup Language (HTML). After publishing the markup language in 1991, and releasing the browser source code for public use in 1993, many other web browsers were soon developed, with Marc Andreessen's Mosaic (later Netscape Navigator), being particularly easy to use and install, and often credited with sparking the Internet boom of the 1990s. It was a graphical browser which ran on several popular office and home computers, bringing multimedia content to non-technical users by including images and text on the same page.

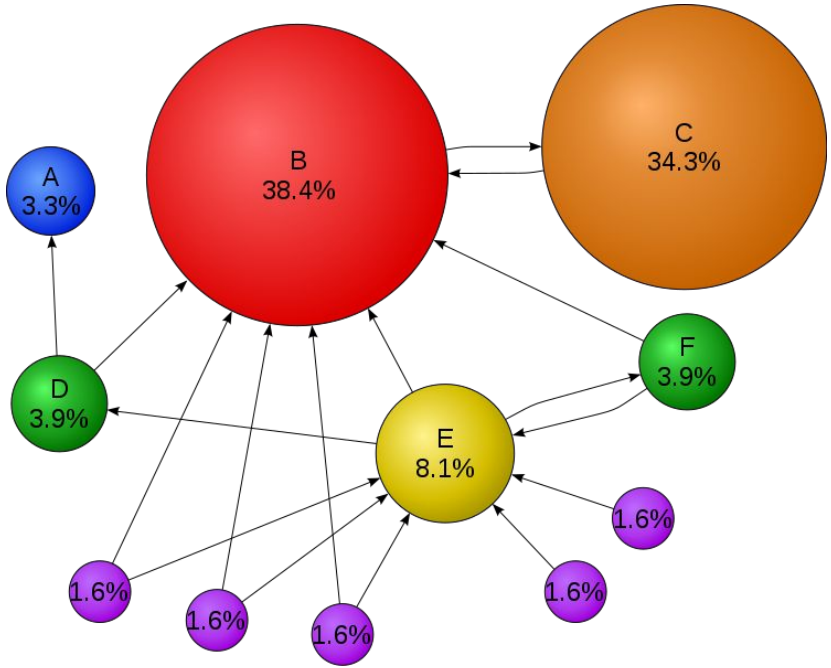
Websites for use by the general public began to emerge in 1993–94. This spurred competition in server and browser software, highlighted in the Browser wars which was initially dominated by Netscape Navigator and Internet Explorer. Following the complete removal of commercial restrictions on Internet use by 1995, commercialization of the Web amidst macroeconomic factors led to the dot-com boom and bust in the late 1990s and early 2000s.

The features of HTML evolved over time, leading to HTML version 2 in 1995, HTML3 and HTML4 in 1997, and HTML5 in 2014. The language was extended with advanced formatting in Cascading Style Sheets (CSS) and with programming capability by JavaScript. AJAX programming delivered dynamic content to users, which sparked a new era in Web design, styled Web 2.0. The use of social media, becoming common-place in the 2010s, allowed users to compose multimedia content without programming skills, making the Web ubiquitous in every-day life.

Википедия – самый наглядный пример успешного применения гиперссылок

PageRank

- Веб можно понимать как огромный связанный гиперссылками *ссылочный граф* (т.н. *веб-граф*)
- Можно использовать ссылочный граф для определения важности страниц
- Идея: чем больше на данную страницу ссылаются другие страницы, тем она, наверное, важнее
- Алгоритм PageRank был изобретен Ларри Пейджем и Сергеем Брином в 1996 г.
- Каждая веб-страница получает *вес* который можно интерпретировать как *вероятность найти случайно блуждающего по графу пользователя на данной странице*
- В 1998 г. вокруг идеи PageRank вырос стартап: Google

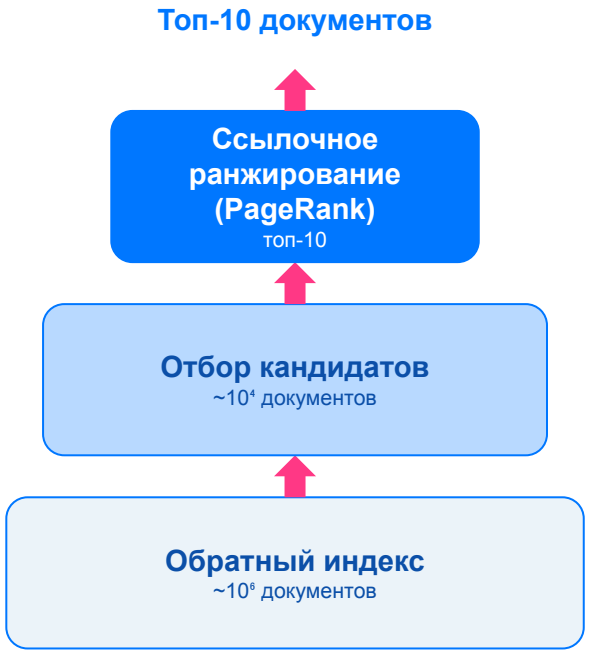


Ссылочный граф и веса веб-страниц согласно алгоритму PageRank

(https://en.wikipedia.org/wiki/PageRank)

Ссылочное ранжирование

- Используем структуру веб-графа для назначения весов (напр. PageRank) веб-страницам
- Ранжируем все найденные по ключевым словам кандидаты с помощью этого веса
- Развитие идеи PageRank: *HITS*, *TrustRank*, ...
- Такой подход был очень популярен в начале 00-х, но сейчас вышел из моды из-за ссылочного спама и централизации интернета
- Мы еще поговорим про все это в лекции №6 про поведенческое и ссылочное ранжирование



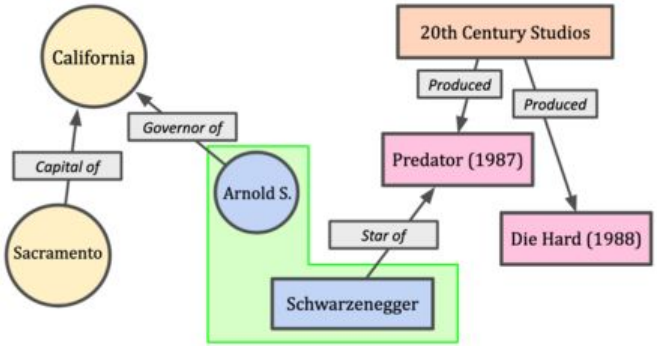
Semantic Web: epic fail?

- В 00-е годы Тим Бернерс-Ли активно продвигал идею *семантического веба* (*semantic web*, или Web 3.0)
- Попытка объединить все данные интернета с помощью семантической разметки в один огромный *граф знаний* (*knowledge graph*)
- Требовал поддержки со стороны веб-мастеров что оказалось его слабым местом
- Тем не менее, графы знаний выжили и используются современными поисковиками (в т. ч. VK), напр. для ответов на вопросы типа «В каком году родился X?»
- К сожалению, не успеем поговорить про них в этом курсе ☹

Пример семантической разметки (https://en.wikipedia.org/wiki/Semantic_Web)

```
<div vocab="https://schema.org/" typeof="Person">
  <span property="name">Paul Schuster</span> was born in
  <span property="birthPlace" typeof="Place" href="https://www.wikidata.org/entity/
Q1731">
    <span property="name">Dresden</span>.
  </span>
</div>
```

Граф знаний (https://en.wikipedia.org/wiki/Knowledge_graph)



SEO

- Вместе с веб-поиском появилась и расцвела целая индустрия поисковой оптимизации (SEO)
- SEO-специалисты реверс-инжинирят алгоритмы ранжирования с целью повышения позиции сайта в поисковой выдаче
- Виды спама: keyword spam, невидимый текст, дорвеи, cloaking, ссылочные фермы, ...



Как выглядит keyword stuffing (взято отсюда)

Как все смешать?

- У нас уже есть много «сигналов» разной природы: текстовый, поведенческий, ссылочный, ...
- Каждый сигнал отражает какой-то аспект релевантности документа запросу
- Например, текстовые скоры (BM25, TF-IDF, ...) показывает насколько документ матчится с запросом по тексту (*lexical matching*)
- CTR отражает популярность среди пользователей
- А PageRank отражает «авторитетность» среди веб-мастеров
- Возникает желание как-то все это объединить...

Формула ранжирования

- В 00-е люди начали делать ручные формулы для смешения разных факторов
- Факторов становилось все больше и больше: категории документов и запросов, гео-привязка, наличие в запросах и документов объектов (имен, названий и т.п.)
- Очень быстро число факторов перевалило за 1000
- Например, в Поиске@Mail.ru у нас было ~5000 факторов

ХРОНОЛОГИЯ АЛГОРИТМОВ ЯНДЕКСА 2007–2023

NN	ДАТА ЗАПУСКА	НАЗВАНИЕ АЛГОРИТМА И ОСНОВНЫЕ ИЗМЕНЕНИЯ
1	2 июля 2007	**«Версия 7»** (новая формула ранжирования, повышение числа факторов, анонс состоялся только на searchengines.guru/showthread.php?t=149644).
2	20 декабря 2007, 17 января 2008	**«Версия 8»** и **«Восьмерка SP1»**. Авторитетные ресурсы получили значимый плюс в ранжировании, внедрение фильтрации «прогонов» для накрутки ссылочных факторов.
3	16 мая, 2 июля 2008	**Магадан** (Fast Rank для быстрого подбора претендентов, мягкость, расширение базы аббревиатур и синонимов, расширенные классификаторы документов), **Магадан 2.0** (уникальность контента, новые классификаторы запросов пользователей и документов).
4	11 сентября 2008	**Находка** (учёт стоп-слов в поисковом запросе, новый подход к машинному обучению, тезаурус).
5	10 апреля, 24 июня, 20 августа, 31 августа, 23 сентября, 28 сентября 2009	**Арзамас** / **Анадырь** (учёт региона пользователя, снятие омонимии), **Арзамас 1.1** (новая региональная формула для ряда городов, кроме Москвы, Санкт-Петербурга и Екатеринбурга), **Арзамас 1.2** (новый классификатор геозависимости запросов), **Арзамас+16** (независимые формулы для 16 регионов России), **Арзамас 1.5** (новая общая формула для геонезависимых запросов), **Арзамас 1.5 SP1** (улучшенная региональная формула для геозависимых запросов).

Релизы формулы ранжирования Яндекса, взято с pixelplus.ru

Ручная формула – это страшно

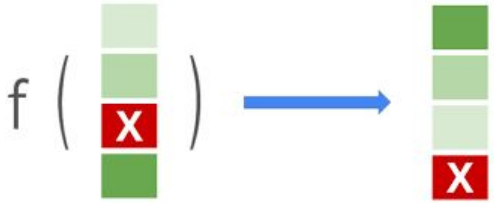
Пример из старого кода поискового движка VK...

```
///Рассчитываем коэффициент для интернет-магазинов, на который будем домножать финальный ранк
F( mf_market_mul_coeff ) = ( !m_site_featurizer.IsMarketSite()
                           || click_rank >= cfg.MarketCrBorder
                           || m_document_featurizer.IsNavigUrl() ? 1 : //Делаем чтобы навигационные запросы не затронуло
                           (1-cfg.MarketMaxPenalty) +
                            cfg.MarketMaxPenalty / (1 + exp(- cfg.MarketSlope *
                                                    ( cfg.MarketBias +
                                                     cfg.MarketPageBRankNormWeight * double(m_document_featurizer.GetMarketPageBRankNorm()) +
                                                     cfg.MarketSiteBRankNormWeight * double(m_site_featurizer.GetMarketSiteBRankNorm()) +
                                                     cfg.MarketPageFixedBRankNormWeight * double(m_document_featurizer.GetMarketPageFixedBRankNorm()) +
                                                     cfg.MarketSiteFixedBRankNormWeight * double(m_site_featurizer.GetMarketSiteFixedBRankNorm()) +
                                                     cfg.MarketCrMixDocAWeight * 1. / ( 1 + exp( - (double(m_qddb_featurizer.GetClickProbability())-cfg.CrMixDocAAvg)/cfg.CrMixDocADeviation ) ) +
                                                     cfg.MarketCrMixDocSWeight * 1. / ( 1 + exp( - (double(m_qddb_featurizer.GetSatisfyProbability())-cfg.CrMixDocSAvg)/cfg.CrMixDocSDeviation ) )
                                                    )
                                                  )
                                          )
                        );

///Рассчитываем коэффициент для мультимедийных запросов для урлов с рекламой, на который будем домножать финальный ранк
F( mf_advert_mul_coeff ) = ( m_document_featurizer.GetAdvert() == 0.f
                           || click_rank >= cfg.AdvertCrBorder
                           || m_document_featurizer.IsNavigUrl() ? 1 : //Делаем чтобы навигационные запросы не затронуло
                           (1-cfg.AdvertMaxPenalty) +
                            cfg.AdvertMaxPenalty / (1 + exp(- cfg.AdvertSlope *
                                                    ( cfg.AdvertBias +
                                                      cfg.AdvertWeight * double(m_document_featurizer.GetAdvertNorm()) +
                                                      cfg.AdvertPageBRankNormWeight * double(m_document_featurizer.GetMarketPageBRankNorm()) +
                                                      cfg.AdvertSiteBRankNormWeight * double(m_site_featurizer.GetMarketSiteBRankNorm()) +
                                                      cfg.AdvertPageFixedBRankNormWeight * double(m_document_featurizer.GetMarketPageFixedBRankNorm()) +
                                                      cfg.AdvertSiteFixedBRankNormWeight * double(m_site_featurizer.GetMarketSiteFixedBRankNorm()) +
                                                      cfg.AdvertCrMixDocAWeight * 1. / ( 1 + exp( - (double(m_qddb_featurizer.GetClickProbability())-cfg.CrMixDocAAvg)/cfg.CrMixDocADeviation ) ) +
                                                      cfg.AdvertCrMixDocSWeight * 1. / ( 1 + exp( - (double(m_qddb_featurizer.GetSatisfyProbability())-cfg.CrMixDocSAvg)/cfg.CrMixDocSDeviation ) )
                                                    )
                                                  )
                                          )
                        );
```

Революция №2: машинное обучение

- Формулу можно и нужно учить
- Задача машинного обучения ранжированию (Learning to Rank, LTR)
- В отличие от привычных задач классификации и регрессии целью является предсказание правильного порядка в рамках запроса
- Широко применяется на практике с конца 00-х годов
- Яндекс использует машинно-обученную формулу с 2009 г. (MatrixNet), а Поиск Mail.ru (VK) — с 2012 г.
- Переход на машинное обучение драматически улучшает качество поиска



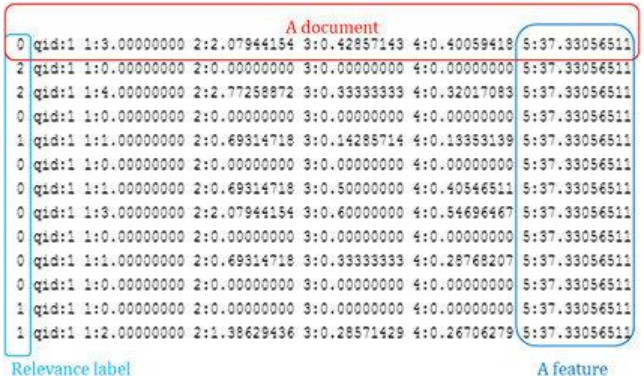
Цель: выучить функцию f(x) которая принимает на вход список объектов и упорядочивает их в порядке убывания релевантности.

Красные квадраты – не-релевантные документы, зеленые – релевантные (чем зеленее тем лучше).

Картинка их статьи Google про TF-Ranking: https://blog.research.google/2021/07/advances-in-tf-ranking.html

Как учить формулу?

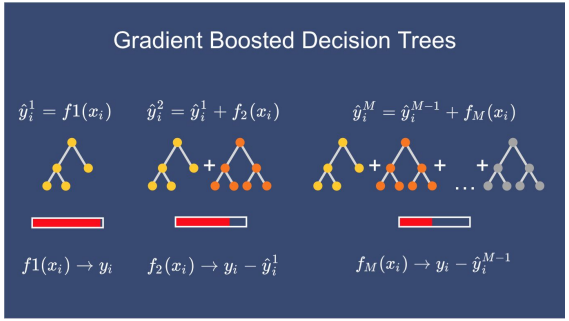
- Датасет = объекты (представленные в виде фичей) + таргеты (ответы)
- Объект – это пара запрос-документ
- Объекты сгруппированы по запросам
- В качестве фичей можно взять классические скоры: BM25, TF-IDF, CTR, PageRank, ...
- Где взять таргет?
- Вариант №1: оцениваем пары запрос-документ асессорами и используем оценку в качестве таргета
- Вариант №2: используем в качестве таргета действия пользователя (клики, просмотры, лайки, ...)
- Обучаем модель предсказывающую правильный порядок
- Классическая задача обучения на табличных данных как на Kaggle!



Как выглядит датасет для машинного обучения ранжированию на примере стандартного датасета LETOR: https://www.microsoft.com/en-us/research/project/letor-learning-rank-information-retrieval/

Алгоритмы Learning to Rank (LTR)

- Теоретическая база заложена в 2005-2010 гг. рисерчерами из Microsoft (Christopher Burges и др.), которые предложили семейство алгоритмов RankNet, LambdaRank, LambdaMART
- Один из лучших на сегодняшний день алгоритмов — YetiRank, был предложен ещё в 2011 г. коллегами из Яндекса (Andrey Gulin, Igor Kuralenok, Dmitry Pavlov)
- Алгоритмы ранжирования хорошо «ложатся» на градиентный бустинг и реализованы в GBDT-фреймворках: CatBoost, XGBoost, LightGBM
- Мы в Поиске VK используем реализацию YetiRank из CatBoost — она, как правило, даёт хороший результат из коробки
- DL-подходы до сих пор не показывают убедительного преимущества над бустингом!*
- Алгоритмам решения задачи LTR будет полностью посвящена лекция N°8



Градиентный бустинг на деревьях, картинка [отсюда]

* См. статью от Google: Qin, Zhen, Le Yan, Honglei Zhuang, Yi Tay, Rama Kumar Pasumarthi, Xuanhui Wang, Mike Bendersky, and Marc Najork. "Are neural rankers still outperformed by gradient boosted decision trees?." (2021). (ссылка)

Пирамида ранжирования

- Ранжирование приобретает вид пирамиды: чем выше уровень => тем меньше документов => тем сложнее модель
- Появляется этап чернового ранжирования, напр. с использованием TF-IDF
- Старые скоры (BM25, CTR, PageRank, ...) становятся признаками!
- Признаки замешиваются в машинно-обученную формулу ранжирования, которая формирует финальную выдачу
- С течением времени слоев становится все больше и больше!



Семантика

- Вернемся обратно к матчингу текстов
- Задача: запрос «*дед мороз*» – как найти документы про Санта Клауса (в которых нет слов «дед мороз»)?
- Традиционные подходы (BM25, TF-IDF) учитывают только точное совпадение ключевых слов (lexical matching)
- Эта проблема называется проблемой *lexical mismatch*
- Хочется отойти от точного совпадения и начать как-то учитывать смысл («*семантику*»)

Синонимы

- Простейший способ добавить семантику – это т.н. *расширение* запроса
- Составляем словарь синонимов и «докидываем» к запросу синонимы
- Это задача решается методами классической вычислительной лингвистики
- Увеличивает полноту поисковой выдачи
- Но и уменьшает точность!
- Еще проблема: надо как-то «балансировать» веса оригинальных ключевых слов и синонимов
- В современном мире этот подход уже не так актуален как раньше

Пример: запрос *«московский международный конкурс юных музыкантов»* превращается в:

(МОСКОВСКИЙ|МОСКВА)
(МЕЖДУНАРОДНЫЙ|ВСЕМИРНЫЙ|ИНТЕРНАЦИОНАЛЬНЫЙ|МИРОВОЙ)
(КОНКУРС|КОНКУРСНЫЙ|ОЛИМПИАДА|СОРЕВНОВАНИЕ|СОСТЯЗАНИЕ)
(ЮНЫЙ|МОЛОДЕНЬКИЙ|МОЛОДОЙ|ЮВЕНИЛЬНЫЙ|ЮНОСТЬ)
(МУЗЫКАНТ|МУЗЫКАНТОВ|АРТИСТ|БАРАБАНЩИК|КОМПОЗИТОР|МУЗЫКАНТСКИЙ|ПИАНИСТ|САКСОФОНИСТ)

Дистрибутивная семантика

- Основана на дистрибутивной гипотезе: *лингвистические единицы, встречающиеся в схожих контекстах, имеют близки смыслы*
- Была предложена Джоном Фертом (John Firth) в 50-е годы
- Представим наш корпус текстов в виде матрицы слово <-> документ
- Применив матричные разложения (напр. SVD), можно представить каждый документ в виде *эмбеддинга* – плотного (dense) вектора из 100-1000 элементов
- Теперь можно выражать семантическую близость как косинусное расстояние между эмбеддингами!
- Такие подходы получили развитие в 90-е годы: Latent Semantic Analysis (LSA)
- На практике работают медленно т.к. надо раскладывать огромные матрицы

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	…
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
…							

Figure 1.1 A term-document incidence matrix. Matrix element (t, d) is 1 if the play in column d contains the word in row t, and is 0 otherwise.

Матрица слово-документ на текстах пьес Шекспира.

Строки соответствуют словам, столбцы – документам.

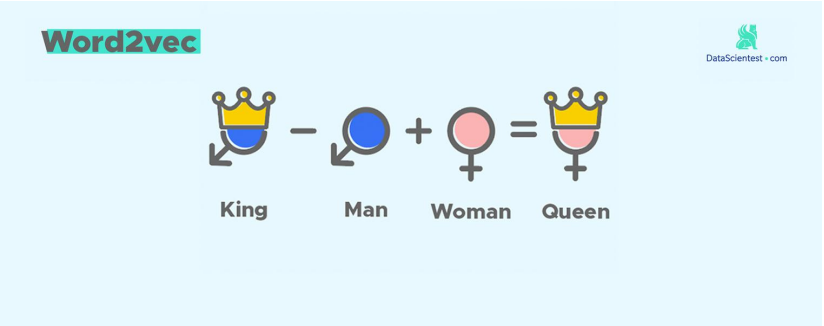
(пример из книги Маннинга)

Дистрибутивная семантика

- Основана на дистрибутивной гипотезе: *слова и выражения, встречающиеся в схожих контекстах, имеют близкие смыслы*
- Была предложена Джоном Фертом (John Firth) в 50-е годы
- Пример:
  - Дед Мороз – это символ Нового Года
  - Санта Клаус – это символ Нового Года
- => «дед мороз» и «санта клаус» имеют близкие значения

Эмбеддинги

- Используя факт встречаемости слов в одинаковых контекстах можно выучить т.н. *эмбеддинги* – плотные (dense) вектора из 100-1000 элементов
- Можно эмбеддить как отдельные слова, так и тексты целиком
- Семантические отношения между текстами представимы в виде простых математических операций над векторами
- Напр., семантическая близость выражается как *косинусное расстояние* между эмбеддингами
- Самая известная модель – *word2vec* была предложена Томасом Миколовым в 2013 г.
- А в 2017 г. вышла популярная библиотека *FastText*



Обязательный пример арифметики над эмбеддингами:

КОРОЛЬ – МУЖЧИНА + ЖЕНЩИНА = КОРОЛЕВА

(картинка взята отсюда)

*Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. (https://arxiv.org/pdf/1301.3781.pdf)

Эмбеддинги в поиске

- Эмбеддинги word2vec или FastText легко применить в поиске для решения проблемы lexical mismatch
- Эмбеддим тексты запросов и документов
- Косинусное расстояние между запросом и документом используем как score, по которому ранжируем:

$$\text{SimilarityScore}(Q, D) = \cos(\text{Emb}(Q), \text{Emb}(D))$$

На практике SimilarityScore просто замешивают как ещё один признак ранжирования, дополняющий TF-IDF.
TF-IDF отвечает за лексический матчинг по ключевым словам, а SimilarityScore — за семантическую близость.

Революция №3: Deep Learning

- В начале 10-х годов произошла революция глубокого обучения (Deep Learning)
- Глубокие нейронные сети начали показывать результаты сравнимые с человеком на задачах распознавания образов
- Революция началась с CV, но к концу 10-х добралась и до NLP

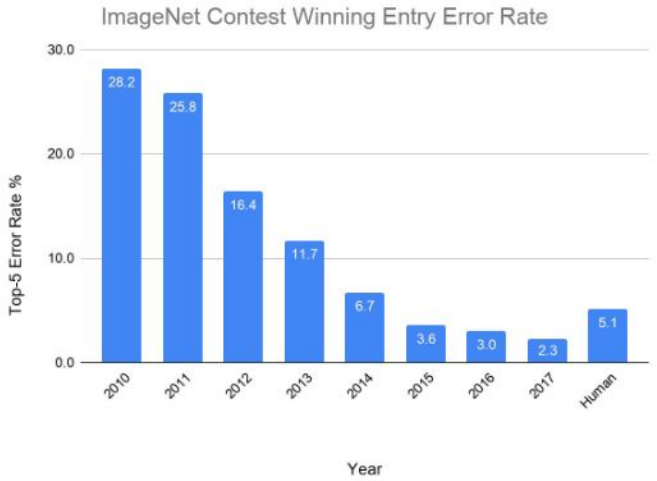


Figure 1: ImageNet classification contest winner accuracy over time

Из статьи Dean, Jeffrey. "1.1 the deep learning revolution and its implications for computer architecture and chip design." *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2020. (https://arxiv.org/ftp/arxiv/papers/1911/1911.05289.pdf)

Модель DSSM

- В 2013 г. исследователи из Microsoft предложили модель DSSM (*)
- Обучается на огромном объеме кликовых данных
- Применяется по отдельности к текстам запроса и документа
- Выдает эмбеддинги, косинусное расстояние между которыми можно использовать в качестве меры семантической близости
- Впервые удалось убедительно побить TF-IDF-подобные модели в задаче текстового ранжирования

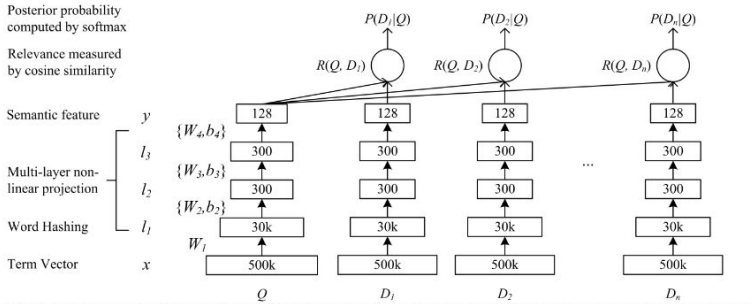


Figure 1: Illustration of the DSSM. It uses a DNN to map high-dimensional sparse text features into low-dimensional dense features in a semantic space. The first hidden layer, with 30k units, accomplishes word hashing. The word-hashed features are then projected through multiple layers of non-linear projections. The final layer's neural activities in this DNN form the feature in the semantic space.

Архитектура DSSM.

В качестве токенов используются буквенные 3-граммы.

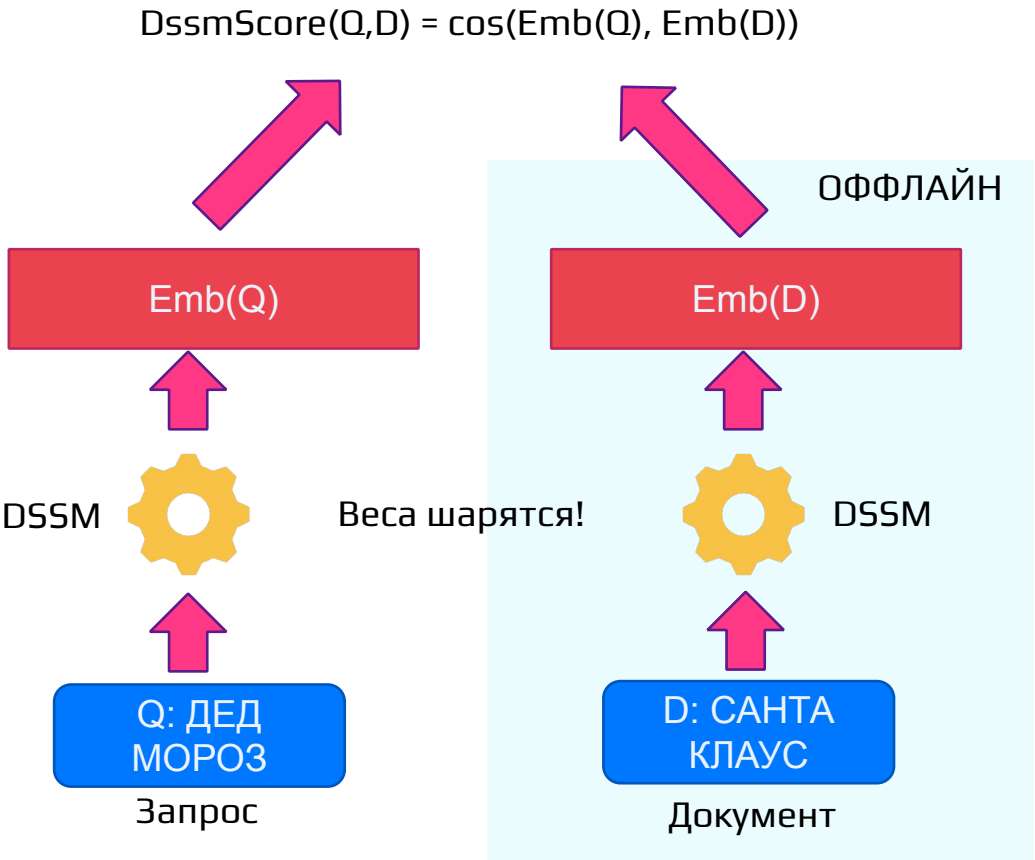
Обучаемся на кликах: предсказываем вероятность клика в документ в поисковой выдаче

Сиамская сеть: запросная и документная часть шерят веса

* Huang, Po-Sen, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. "Learning deep structured semantic models for web search using clickthrough data." In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*, pp. 2333-2338. 2013. (скачать)

Би-энкодеры в поиске

- Модель DSSM – это т.н. **bi-encoder**: применяется к текстам запроса и документа ПО ОТДЕЛЬНОСТИ
- Би-энкодеры позволяют перенести большую часть вычислений с онлайна в оффлайн
- В *офлайне* (т.е. во время индексации) предрассчитываем эмбеддинги всех документов и сохраняем в поисковом индексе
- В *рантайме* (т.е. во время обработки запроса) эмбеддим запрос и вычисляем косинусное расстояние с каждым из найденных в индексе документов



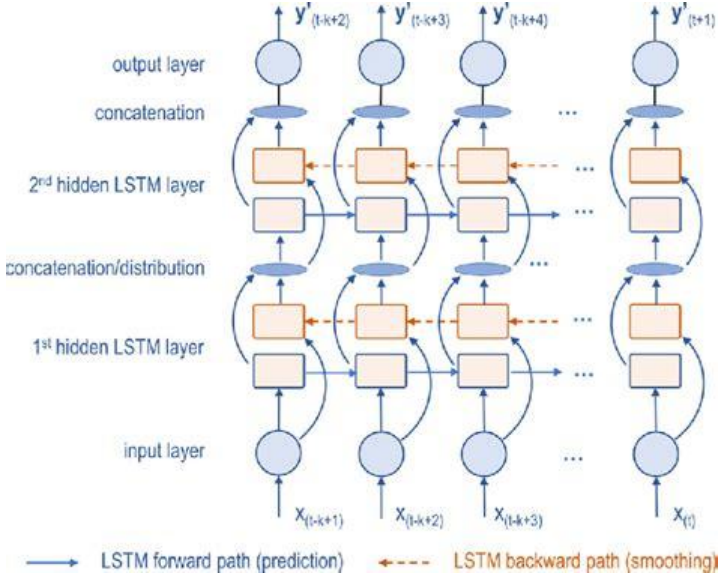
Neural Information Retrieval

- К концу 10-х годов появилось множество нейронок для семантического матчинга: C-DSSM, CLSM, DRRM, K-NRM, …
- Все полученные таким образом скоры замешиваются в модель ранжирования в качестве отдельных признаков («нейрофичи») — и это очень мощные признаки!
- Вся эта деятельность получила название Neural Information Retrieval («нейропоиск»)
- Мы тоже внедрили DSSM-подобные модели в поиске VK в 2019 г.
- Подробнее поговорим про задачу семантического матчинга в лекции №9



Недостатки BoW-моделей

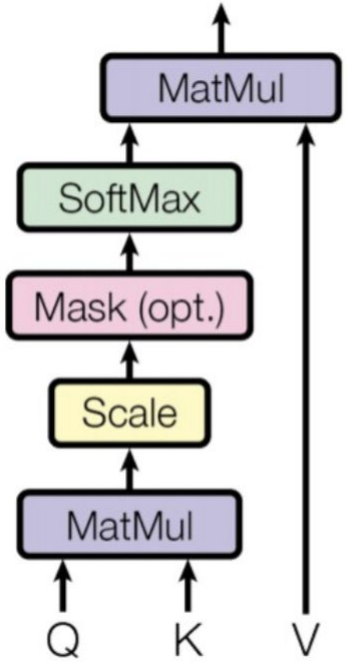
- DSSM, FastText и подобные модели – это по своей сути bag of words модели, т.е. не учитывают порядок слов
- А для обработки текста порядок слов, очевидно, очень важен
- Первые попытки его учитывать были основаны на рекуррентных нейронных сетях, таких как LSTM
- Но LSTM работали медленно и не давали убедительного роста скоров



Сложный 2-слойный и двусторонний LSTM

Наступил 2017 г. ...

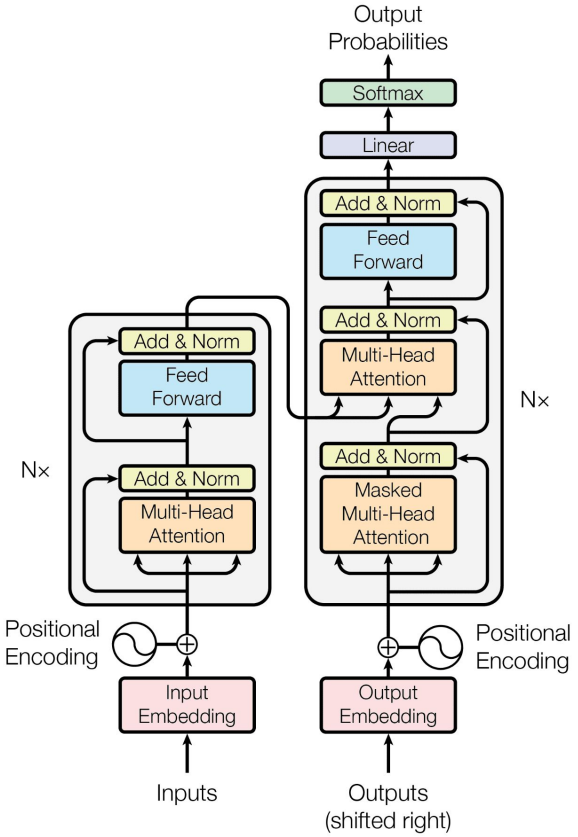
- Вопрос: что случилось в 2017?



Картинка-подсказка

Революция №4: трансформеры

- В 2017 появились модели на архитектуре Трансформер (*)
- Модели стали по настоящему большими: >100 млн. и более параметров
- Драматически выросло качество на задачах связанных с обработкой последовательностей, в т.ч. задачах NLP (классификация, суммаризация, машинный перевод, NER, генерация текста, ...)
- Не остались в стороне и задачи поиска

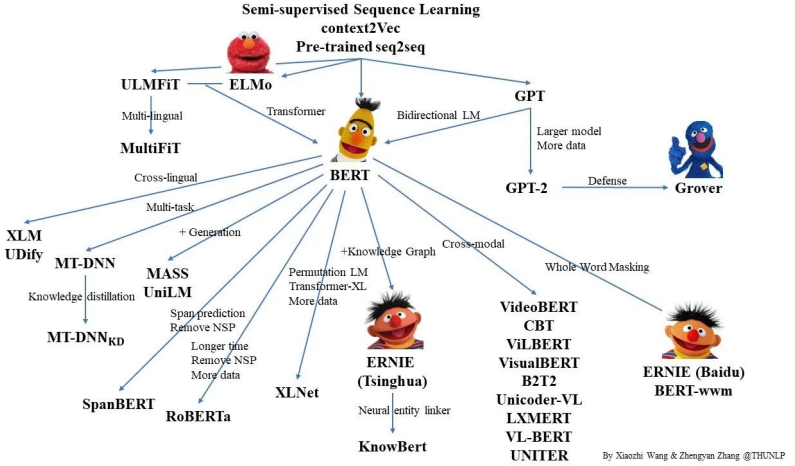


Архитектура Трансформер (картинка с сайта https://machinelearningmastery.com/the-transformer-model/)

* Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*. (ссылка)

BERT

- В 2018 г. появилась модель BERT (и его потомки) – «рабочая лошадка современного» NLP
- BERT – это целое семейство моделей
- BERTы обучаются на задачу предсказания слов по их контексту (MLM, masked language modeling) – развитие идей дистрибутивной семантики!
- BERTы выдают *контекстно-зависимые* эмбеддинги – мы теперь учитываем порядок слов
- «Ванильный» BERT в современном мире используется редко
- В VK широко используем модель *XLM-RoBERTa* (*) которая хорошо «понимает» русский язык
- BERTы можно применить для задачи текстового ранжирования



Семейство BERTов по состоянию на 2020 г.

* Conneau, Alexis, et al. "Unsupervised cross-lingual representation learning at scale." *arXiv preprint arXiv:1911.02116* (2019). (https://arxiv.org/pdf/1911.02116.pdf)

BERTы в поиске

- Для каждой пары запрос-документ считаем скор *BertScore(Q,D)* который используем для ранжирования документов
- По сути, все аналогично DSSM, но работает гораздо лучше!
- Модели очень большие => приходится брать готовые претрейны и файнтюнить под наши датасеты
- Мы выкатили первый трансформер в поиске VK в прод в 2021 г. (в качестве претрейна использовалась XLM-RoBERTa)
- А в 2023 г. у нас появились собственные претрейны!
- На данный момент трансформеры широко используются в задаче текстового ранжирования, напр. Google использует модель RankT5 на базе модели T5 (*)

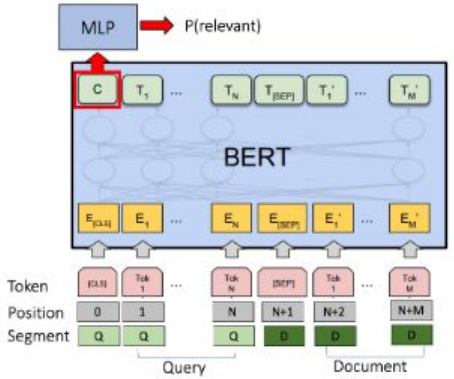
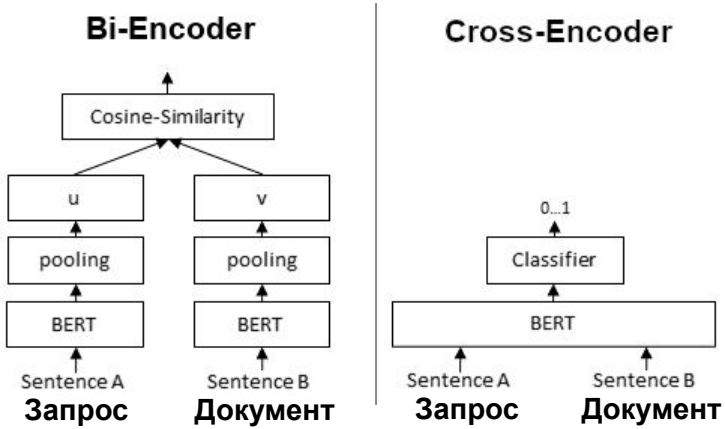


Figure 1: BERT sentence pair classification architecture [3].

* Zhuang, Honglei, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. "Rankt5: Fine-tuning t5 for text ranking with ranking losses." *arXiv preprint arXiv:2210.10634* (2022). (https://arxiv.org/pdf/2210.10634.pdf)

BERTы – это сложно

- Проблема №1: BERT – это кросс-энкодер, т.е. принимает на вход тексты СРАЗУ и запроса, и документа
- Проблема №2: BERT работает медленно
- Надо как-то превратить BERT в би-энкодер
- Надо ускорять инференс с помощью дистилляции, квантизации, применения эффективных фреймворков для инференса и т.д.
- Выкатить такую модель в прод – целое приключение
- Про все это поговорим подробнее в лекции №10 про ранжирование с помощью нейронок



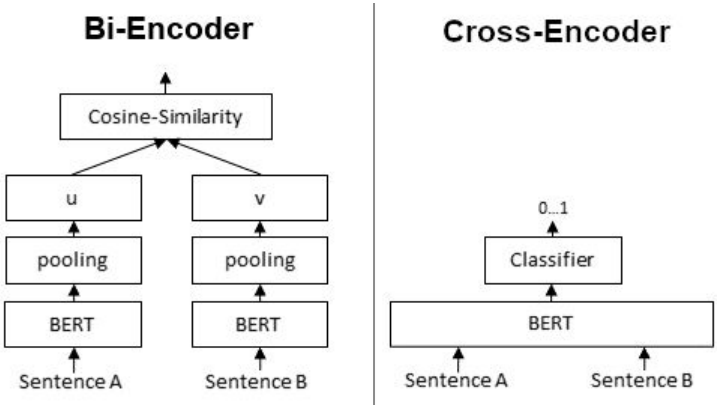
Отличие кросс-энкодера от би-энкодера.

Кросс-энкодер принимает на вход тексты СРАЗУ и запроса, и документа.

Би-энкодеры могут применяются к документам и запросам ПО ОТДЕЛЬНОСТИ => эмбеддинги документов можно предрасчитать в оффлайне!

Как обучать BERT?

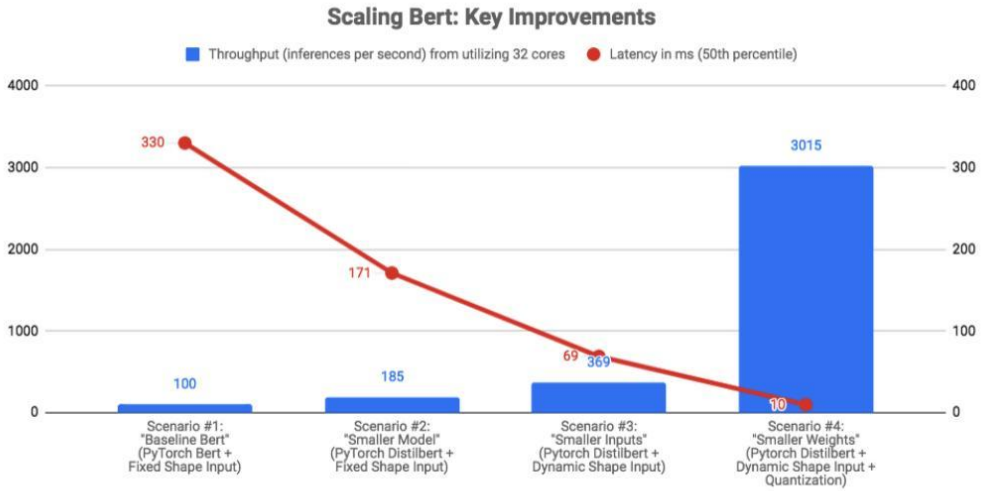
- Проблема №1: BERT – это кросс-энкодер, т.е. когда мы обучаем BERT под задачу текстового матчинга, он принимает на вход тексты СРАЗУ и запроса, и документа
- До сих пор мы имели дело только с би-энкодерами (такими как DSSM), которые могут применяются к документам и запросам ПО ОТДЕЛЬНОСТИ
- Кросс-энкодеры не позволяют предрасчитать эмбеддинги документов на этапе индексации => их нельзя применить в поиске!
- Надо как-то превратить BERT в би-энкодер



Отличие кросс-энкодера от би-энкодера

Как инференсить BERT?

- BERT-Large содержит ~340 млн. параметров, а T5-Large ~770 млн. параметров
- Проблема №2: такие модели инференсятся очень медленно, особенно на CPU!
- А GPU очень дорогие ☹
- Поисковый бэкенд должен отвечать на запросы пользователя быстрее 100 мс
- А во время индексации мы должны заэмбеддить >10^9 документов
- Выкатить такую модель в прод – это целое приключение

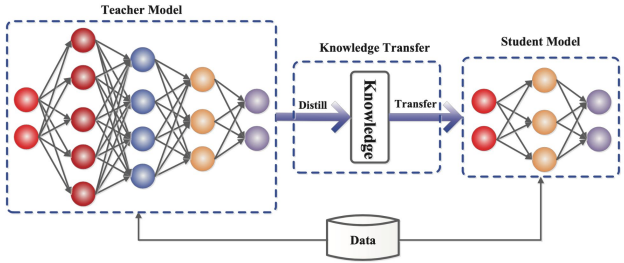


Типичные времена инференса ванильного BERTa >100 мс

(взято из блога компании Roblox)

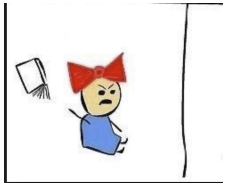
Рецепт выкатки BERTa от Поиска VK: начало

- Берем претрейн XLM-RoBERTa: обученный кем-то или самостоятельно
- XLM-RoBERTa – это *кросс-энкодер:* получает на вход тексты СРАЗУ и запроса, и документа, и выдает score – насколько документ релевантен запросу
- Файн-тюним кросс-энкодер на поисковом датасете под задачу текстового ранжирования
- *Дистиллируем* кросс-энкодер в маленький (2 или 4 слоя энкодера) *би-энкодер*, который можно применять к запросам и документам ПО ОТДЕЛЬНОСТИ
- Би-энкодер выдает эмбеддинги, косинусное расстояние между которыми можно использовать в качестве нашего score релевантности



Дистиллируем большую модель-учитель в маленькую модель-ученик

Рецепт выкатки BERTa от Поиска VK: конец



- Теперь у нас есть маленький би-энкодер, осталось выкатить его в прод
- (Опционально) Квантуем би-энкодер в int8 чтобы быстрее инференсить прилетающие от пользователей запросы
- Используем эффективный фреймворк для инференса, напр. **onnxruntime** или **tensorrt**
- (Опционально) Если все равно недостаточно быстро, то ставим дорогие GPU
- Предрассчитываем эмбеддинги всех документов (все как для DSSM!)
- (Опционально) *Квантуем* (quantization) эмбеддинги документов, чтобы они занимали меньше места в индексе
- Так мы добиваемся времени инференса <10 мс!
- Про все это поговорим подробнее в лекции №10 про ранжирование с помощью нейронок

Отбор кандидатов revisited

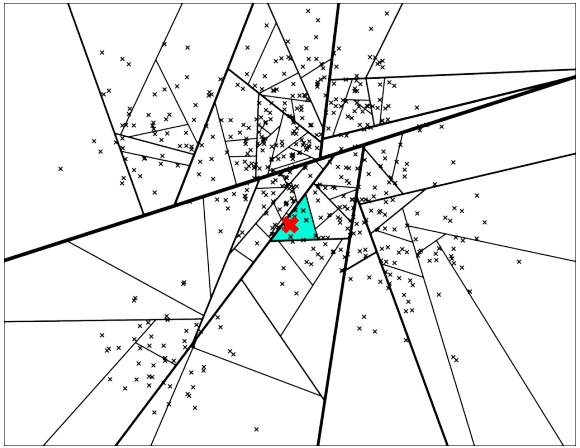
- Вернемся к обратному индексу
- Там у нас до сих пор работает булев поиск!
- Булев поиск не умеет семантический матчинг => мы теряем множество хороших документов еще на первом этапе (напр. документы про Санта Клауса по запросу «дед мороз»)
- Можем ли мы применить для отбора кандидатов семантический матчинг с использованием эмбеддингов?

Теряем хорошие документы!
булев поиск не умеет семантический матчинг —
документы про Санта-Клауса по запросу «дед мороз»
не доедут даже до ранжирования



Векторный поиск

- Хотим находить документы, эмбеддинги которых наиболее близки к эмбеддингу запроса
- И делать это быстро даже в базе из миллиардов эмбеддингов!
- Нам поможет т.н. *приближенный поиск ближайших соседей* (ANN, approximate nearest neighbor search)
- Самое популярное решения: библиотека **faiss**
- Мы в поиске VK используем библиотеки **annoy** и **hnswlib**
- Такой поиск часто называют *векторным поиском*, а базу, в которой лежат эмбеддинги документов – *векторным индексом*
- Векторному поиску будет посвящена лекция №11



Визуализация работы библиотеки **annoy** (из блога Eric Bernhardsson)

Поиск сегодня

- Современная поисковая система выглядит как-то так
- Каждая новая идея наслаивается на предыдущие!
- Рядом работают и TF-IDF из 1970-х, и LTR из 2000-х, и трансформеры из 2020-х

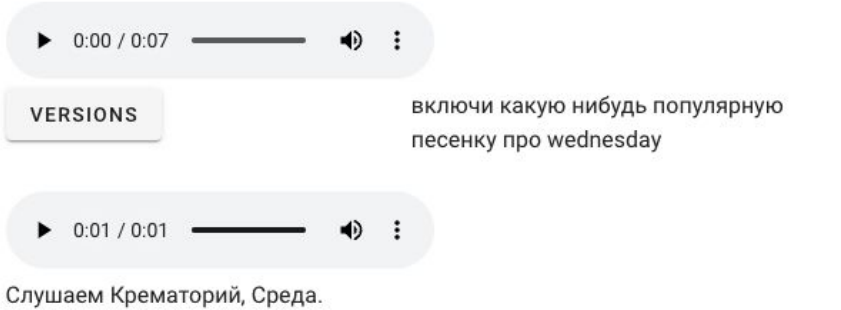


Революция №5: LLM

- В ноябре 2022 OpenAI запустил ChatGPT — и всё завертелось
- В 2023 утекли веса Llama, появилось множество открытых моделей
- Взрывной рост числа параметров, потом — рост «рассуждающих» моделей
- К 2025–2026 LLM переехали внутрь самих поисковиков: генеративные ответы стали частью выдачи, а не отдельным продуктом
- Мы в VK тоже учимся учить и применять LLM
- Так убьют ли LLM поиск? Мешают две вещи: галлюцинации и то, что знания модели «заморожены» на момент обучения

Проблема актуальности информации

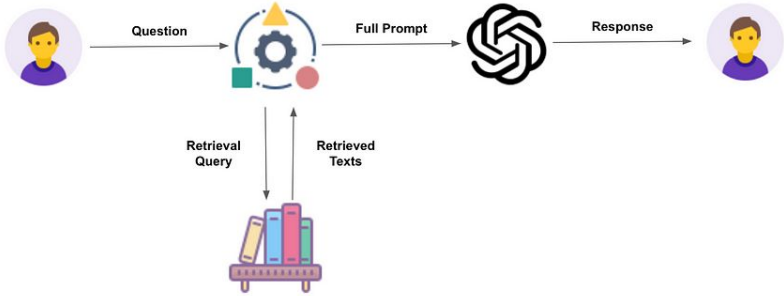
- В мире постоянно появляется что-то новое
- Знания модели «заморожены» на момент когда был собран датасет на котором она училась
- Переобучение модели – это долгий и ОЧЕНЬ дорогой процесс
- Сложно поддерживать модель в актуальном состоянии



Маруся демонстрирует отличное знание английского, но ничего не знает про сериал Wednesday ☹

RAG

- Можно использовать связку LLM + Поиск для того чтобы найти и «скормить» модели свежую и фактологически верную информацию
- При таком подходе LLM не заменяет, а дополняет поиск!
- Пока кажется что будущее за гибридами LLM + Поиск
- Такой подход называется RAG (*Retrieval Augmented Generation*)



LLM пользуется поиском

Ранжирование LLMами

- Даже в будущем останется потребность в «обычном» поиске, когда пользователь хочет получить ранжированную выдачу по запросу
- Можно ли применить LLMы для решения задачи ранжирования?
- Конечно! Например, мы можем просто попросить LLM отранжировать документы в правильном порядке
- Это по сути еще один слой в пирамиде ранжирования
- Уже сейчас такой подход показывает результаты лучше чем BERTы, но работает невероятно медленно...
- Stay tuned!

```
<|system|>
You are RankLLM, an intelligent
assistant that can rank passages based on
their relevancy to the query.
<|user|>
I will provide you with {num} passages,
each indicated by a numerical identifier [].
Rank the passages based on their relevance to
the search query: {query}.

[1] {passage 1}
[2] {passage 2}
...
[{num}] {passage {num}}

Search Query: {query}.

Rank the {num} passages above based on their
relevance to the search query. All the
passages should be included and listed
using identifiers, in descending order of
relevance. The output format should be [] > [],
e.g., [4] > [2]. Only respond with the ranking
results, do not say any word or explain.
<|assistant|>
```

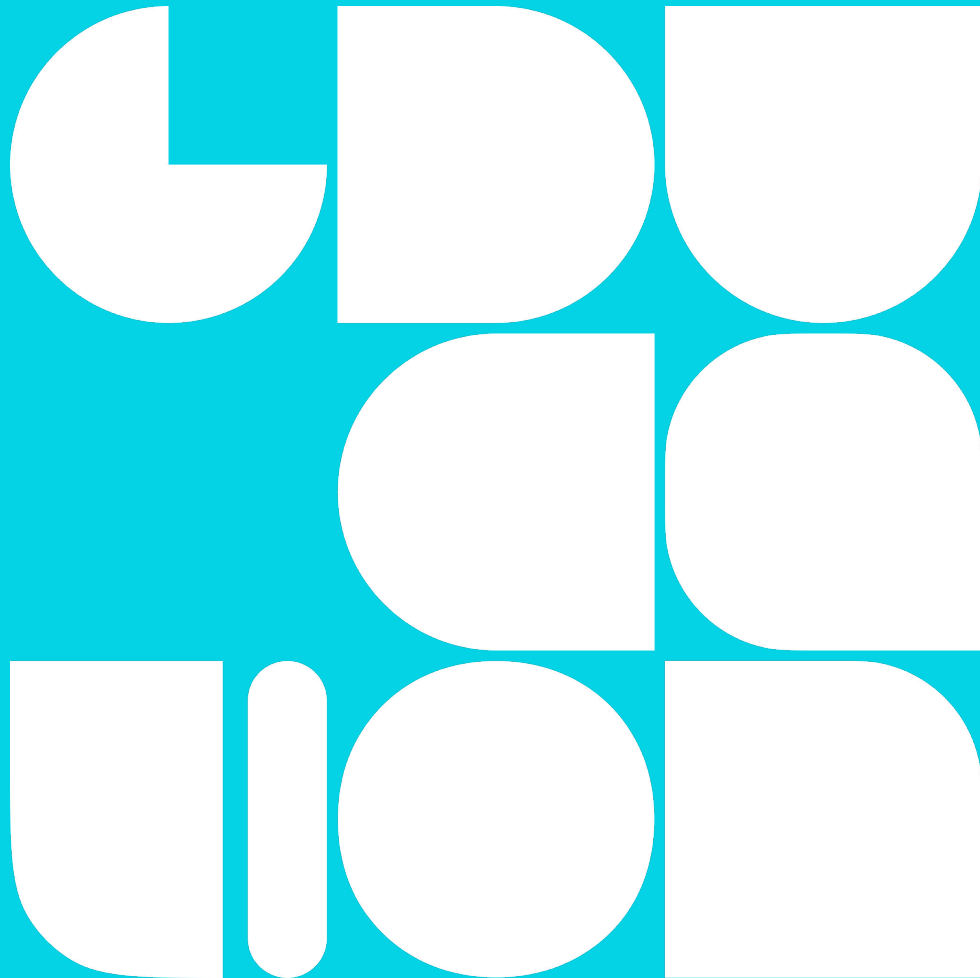
Model Generation: [9] > [4] > [20] > ... > [13]

Figure 1: The reranking prompt and sample generation for RankZephyr.

Дорогой Zephyr, отранжируй пожалуйста документы в правильном порядке...

Pradeep, Ronak, Sahel Sharifymoghaddam, and Jimmy Lin. "RankZephyr: Effective and Robust Zero-Shot Listwise Reranking is a Breeze!." (https://arxiv.org/pdf/2312.02724.pdf)

Архитектура современного поисковика

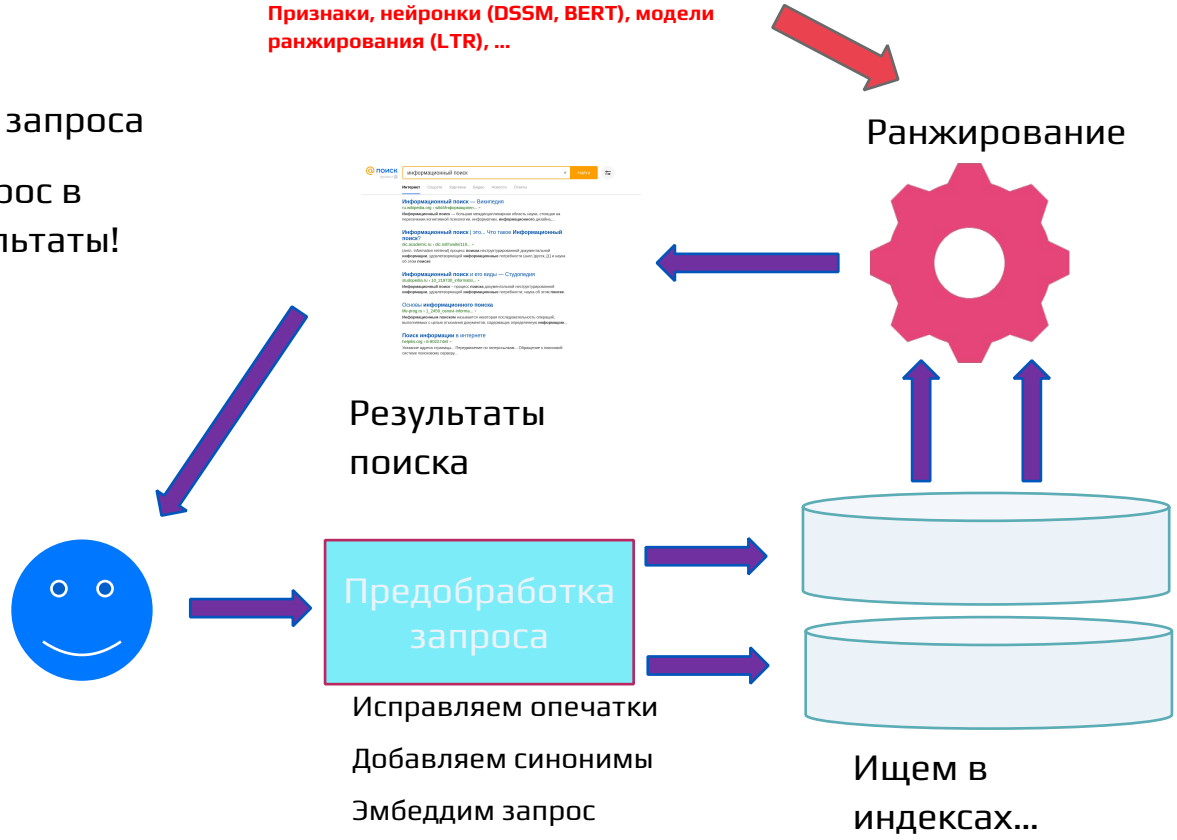


Мы уже почти все знаем!

- До сих пор мы рассматривали поиск с «концептуальной» точки зрения
- Теперь посмотрим как все это выглядит с инженерной
- Будем разделять *оффлайн* и *онлайн*-архитектуру
- Онлайн – это все что касается обработки поискового запроса
- Оффлайн – это то, чем ЕЩЕ занимается поисковик: подготовка данных, индексация, расчет эмбеддингов и т.п.

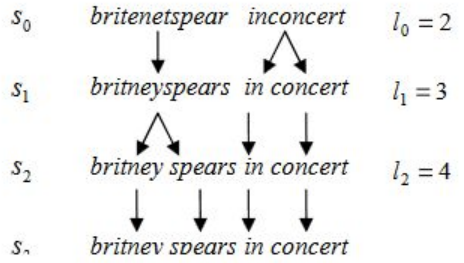
Онлайн-архитектура поиска

- Проследим путь поискового запроса
- Все очень просто: ищем запрос в индексах и ранжируем результаты!



Предобработка запросов

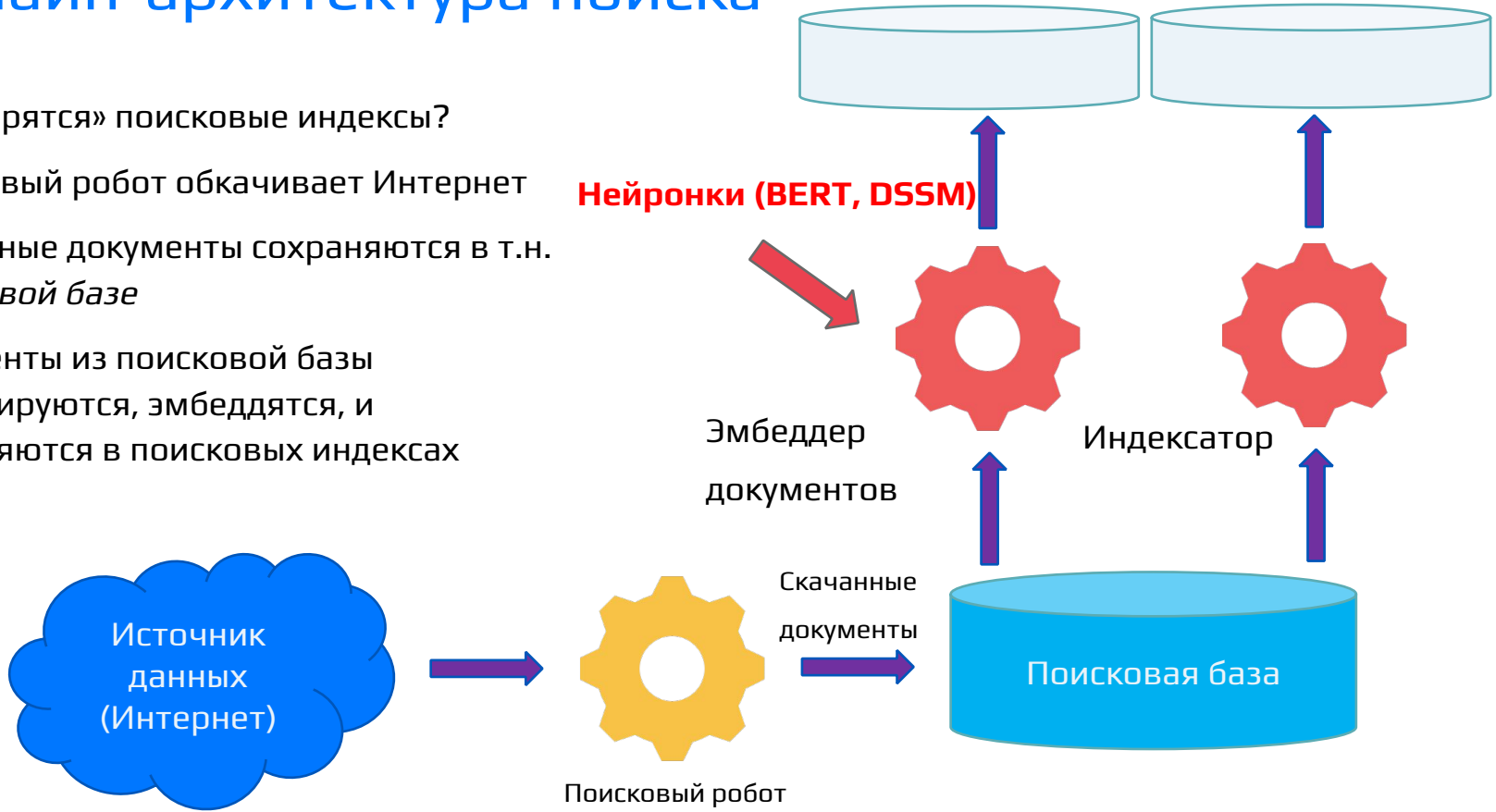
- Мы уже говорили про эмбеддинги и расширение запроса синонимами
- Еще одна важная тема – это исправление опечаток
- Подробно поговорим про это в лекции №3



Исправляем поисковые запросы, пример из классической статьи Cucerzan, Silviu, and Eric Brill. "Spelling correction as an iterative process that exploits the collective knowledge of web users." *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. 2004. (ссылка)

Оффлайн-архитектура поиска

- Как «варятся» поисковые индексы?
- Поисковый робот обкачивает Интернет
- Скачанные документы сохраняются в т.н. *поисковой базе*
- Документы из поисковой базы индексируются, эмбеддятся, и сохраняются в поисковых индексах



Поисковые база и робот

- Эффективно обкачивать Интернет — это сложная инженерная задача
- Ей занимается т. н. поисковый робот (он же краулер, он же спайдер)
- Скачанные данные надо хранить и обрабатывать
- Речь идёт о многих петабайтах данных!
- В Поиске VK и хранилищем, и средой распределённой обработки данных для нас выступает YT
- Про поисковый робот и краулинг поговорим отдельно

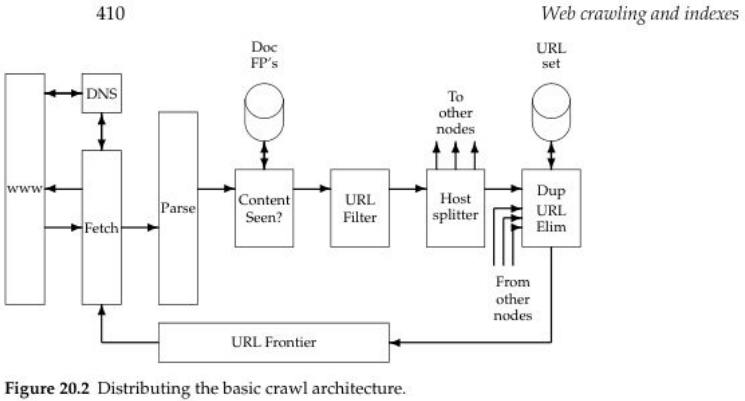


Figure 20.2 Distributing the basic crawl architecture.

Архитектура распределенного краулера (картинка из книги Маннинга)

Поиск в 2026: генеративный слой

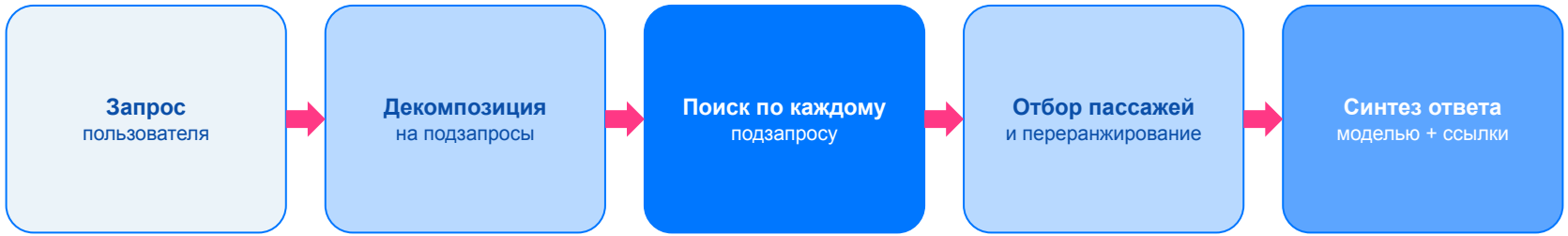
За три года **генеративный ответ** прошёл путь от демо до основного интерфейса поиска

- Google: сначала AI Overviews прямо в выдаче, потом отдельный AI Mode. На I/O 19 мая 2026 объявили, что за год после запуска AI Mode перевалил за 1 млрд пользователей в месяц, а моделью по умолчанию стала Gemini 3.5 Flash
- Яндекс: Нейро на YandexGPT (2024) вырос в «Поиск с Алисой» — на часть запросов вместо списка ссылок пользователь получает синтезированный ответ из нескольких источников со ссылками
- Ключевое: это не замена поиска нейросетью. Это **ещё один слой НАД поиском**
- Классическая выдача никуда не делась — она стала входом для модели, а не конечным результатом
- Появился новый жаргон — GEO / AEO: оптимизируют уже не позицию в топе, а попадание в цитируемые источники ответа

Как устроен генеративный ответ

Под капотом — RAG поверх обычного поиска. По шагам:

- Запрос декомпозируется на подзапросы (query fan-out): «сравни X и Y» превращается в десяток отдельных поисков
- По каждому подзапросу отрабатывает обычный поиск, а модель уже читает топ и собирает из пассажей ответ со ссылками



↑ **здесь работает вся пирамида ранжирования:**
обратный и векторный индексы, отбор кандидатов, LTR

Один «ответ» = десятки обращений к поиску. Классика не отменяется — она просто ушла под капот.

Что генеративный слой меняет для поиска

- Единица выдачи: ранжируем уже не документы, а пассажи. Важно, чтобы кусок текста был самодостаточным и цитируемым
- Цена ошибки: одна нерелевантная ссылка в топе — мелочь, один непроверенный факт в ответе — галлюцинация с вашим логотипом
- Метрики: nDCG по документам плохо описывает качество ответа. Нужны grounding (всё ли подтверждено источником), полнота и отсутствие противоречий
- Латентность: вместо одного поиска — десяток параллельных плюс генерация. Бюджет растягивается с сотен миллисекунд до секунд
- Разметка: асессор теперь оценивает не «релевантен ли документ», а «правда ли это и подтверждено ли источником»
- Экономика: пользователь всё чаще не кликает. Для площадок это падение трафика, для поисковика — вопрос, чем платить за инференс

Агентный поиск: от ответа к задаче

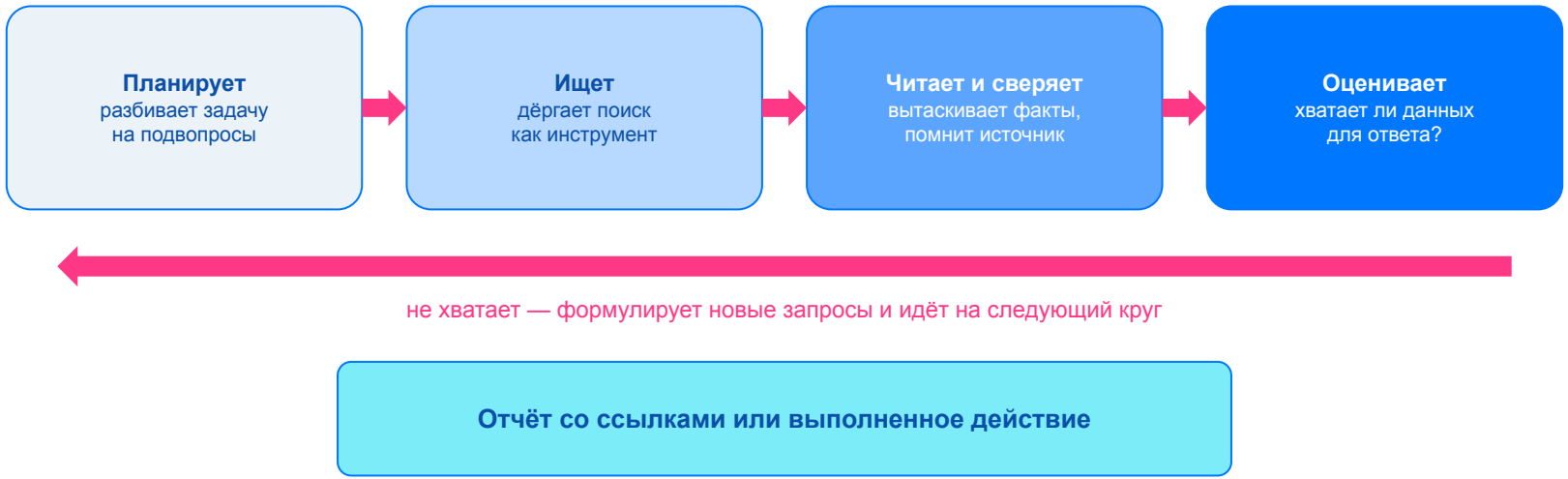
Удобно смотреть как на лестницу — от «поиск отвечает» к «поиск делает»

- Уровень 0 — классическая выдача: список ссылок, всё дальше решает человек
- Уровень 1 — генеративный ответ: один проход RAG, ответ на один вопрос
- Уровень 2 — deep research: агент сам планирует десятки запросов, читает источники, сверяет их и пишет отчёт. Минуты работы вместо секунд
- Уровень 3 — фоновые агенты: на I/O 2026 Google показал информационных агентов, которые работают круглосуточно, следят за изменениями по вашей задаче и присылают сводку
- Уровень 4 — действия: забронировать, сравнить и купить, позвонить в организацию, собрать мини-приложение под задачу прямо в выдаче
- Граница проходит между 1 и 2: дальше поиск перестаёт отвечать и начинает выполнять задачу

Как работает поисковый агент

Внутри агента не один проход, а цикл — и это главное отличие от обычного RAG

- Останавливается по бюджету: числу шагов, времени или уверенности. Иначе будет крутиться вечно



Что это значит для инженера поиска

- Главным потребителем выдачи становится не человек, а модель. Ей не нужны красивые сниппеты — нужны точные самодостаточные куски текста и честные метаданные
- **Поиск превращается в API**: ключевой интерфейс — не страница выдачи, а инструмент, который агент вызывает десятки раз за одну задачу
- Растут требования к свежести: агент, который следит за изменениями, бесполезен на индексе недельной давности
- Растёт роль переранжирования LLM-ками — про это будет отдельная лекция в курсе
- И при этом ничего из классики не отменяется: обратный индекс, BM25, LTR, векторный поиск по-прежнему работают внутри. Просто вызывает их теперь не пользователь, а агент
- Возможно, разберём это подробнее в конце курса — если будет интерес

Итоги

Сегодня на занятии разобрали:

- Орг. вопросы
- Что такое инфопоиск и каким он бывает
- Историю инфопоиска и его основные эволюции
- Нейронки в поиске и их перспективы
- Куда всё едет сейчас: генеративный слой над поиском и агентный поиск

Вопросы?



education

Материалы для подготовки: книги

- Маннинг К. Д., Рагхаван П., Шютце Х. «Введение в информационный поиск», 2011 — основной учебник курса
- Büttcher S., Clarke C. L. A., Cormack G. V. Information Retrieval: Implementing and Evaluating Search Engines, 2016
- plg.uwaterloo.ca/~ir/ir/book/ — хорошо дополняет Маннинга: раскрыты темы, которых у него нет, например BM25
- Croft W. B., Metzler D., Strohman T. Search Engines: Information Retrieval in Practice, 2010

Материалы для подготовки: видео и статьи

- Видеолекции Техносферы (2017): youtube.com/watch?v=drphXdTSj3w — плейлист по классическому инфопоиску
- arxiv.org/list/cs.IR/recent — свежие статьи по информационному поиску, 5–15 в день
- Конференции, за которыми стоит следить: SIGIR, CIKM, WSDM, ECIR
- Ссылки на все статьи, упомянутые на лекциях, есть в сносках на слайдах

Спасибо за внимание!